

Motion Planning with Differential Constraints

TSFS12: Autonomous Vehicles
Planning, Control, and Learning Systems
Lecture 4

Purpose of this Lecture

- Give a background on **methods for motion planning** of systems with **differential motion constraints**.
- Study how **rapidly-exploring random trees (RRTs)** and extensions of such strategies can be used for motion planning.
- **Extend** the graph-search methods from Lecture 2 to autonomous vehicles with **differential motion constraints** using **lattice planning**.
- *Extra material:* Study how **path-constrained trajectory-planning problems** can be solved.

Expected Take-Aways from this Lecture

- Knowledge about how important **differential motion constraints** can be considered in motion planning.
- Be familiar with commonly used **methods for motion planning** targeting **autonomous vehicles**.
- Knowledge about some available **computer tools and libraries** for motion planning.

Literature Reading

The following book and article sections are the main reading material for this lecture. References to further reading are provided throughout the slides and at the end of the lecture slides.

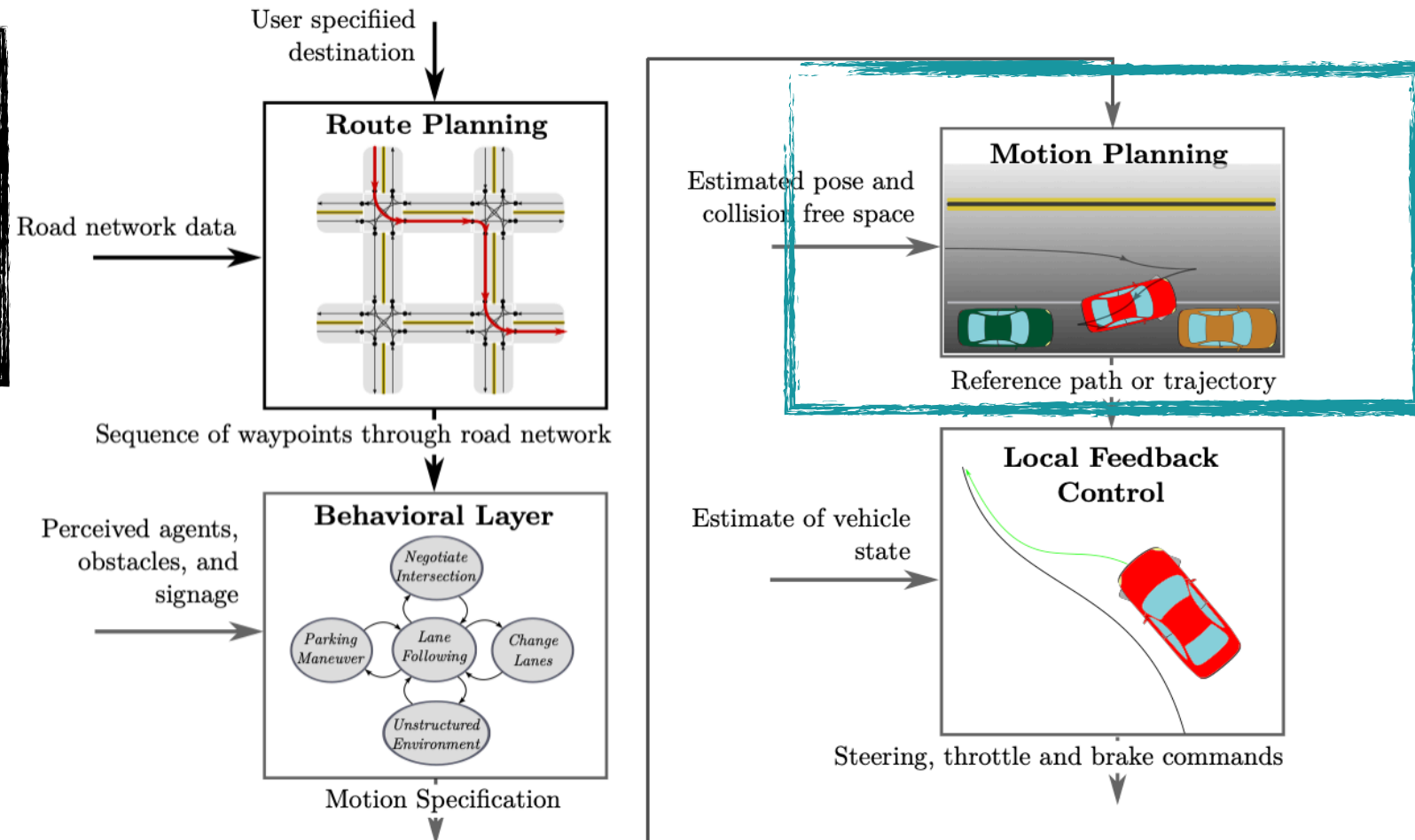
- Sections 5.1-5.5 and 14.1-14.4 in LaValle, S. M: *Planning Algorithms*. Cambridge University Press, 2006.
- Sections 3.3.3 and 5 in Karaman, S., & E. Frazzoli: "Sampling-based algorithms for optimal motion planning". *The International Journal of Robotics Research*, 30(7), 846-894, 2011.
- Section 4.5 in Bergman, K: "Exploiting Direct Optimal Control for Motion Planning in Unstructured Environments ", Ph.D. Thesis No. 2133, Div. Automatic Control, Linköping Univ., 2021.

Outline of the Lecture

- The **motion-planning problem**.
- Methods:
 - **Rapidly-exploring random trees** (RRTs) for motion planning.
 - Planning using **motion primitives** and **state lattices**.
- **Tools and libraries** for motion planning.
- *Extra material:* **Decoupled approaches** to motion planning.

Context in the Architecture for an Autonomous Vehicle

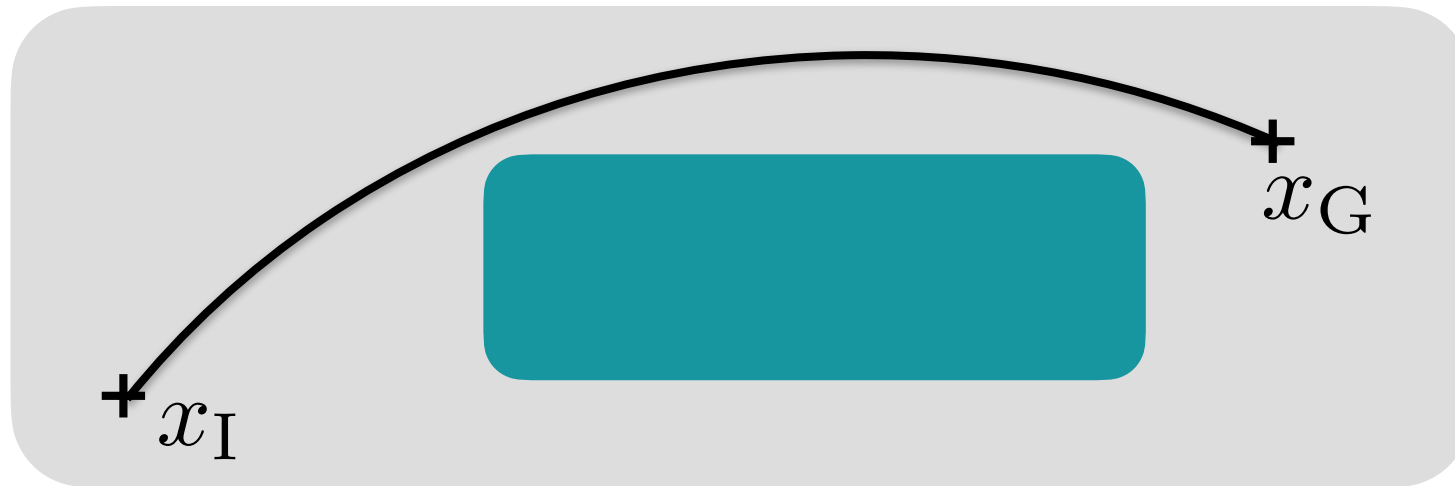
In this lecture the focus will be on the motion-planning layer in the decision-making architecture.



The Motion-Planning Problem

Motion Planning – Introduction (1/3)

- Compute a strategy for transferring a vehicle **from an initial state to another desired state**.
- The state could be the **position, orientation, and derivatives thereof**, for the vehicle.



Motion Planning – Introduction (2/3)

- Constraints to be fulfilled during the motion (**feasibility**):
 - The **motion equations** of the vehicle.
 - **Obstacles, humans, and other vehicles** (stationary or time-varying).
 - **Physical limits** on control inputs (e.g., motor power in a car, maximum torque in a robot motor).

Motion Planning – Introduction (3/3)

- Constraints to be fulfilled during the motion (**feasibility**), cont'd:
 - Constraints on **internal states** (e.g., velocity, available friction between tires and road, torque transfer in clutches).
 - **Road constraints.**
- Often also desirable to **minimize** energy/power consumption, time, path length, or other performance criteria (**optimality**).

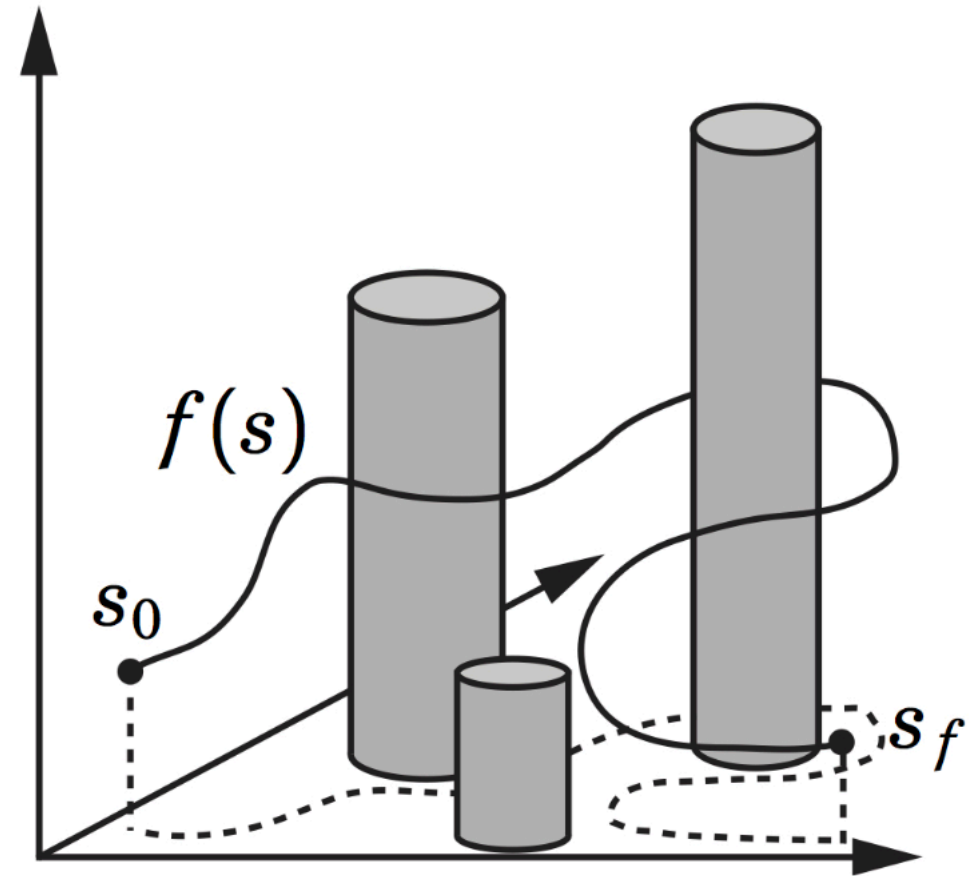
Some Important Terms and Concepts

- **Path:** A **curve in space** defining the geometric motion.

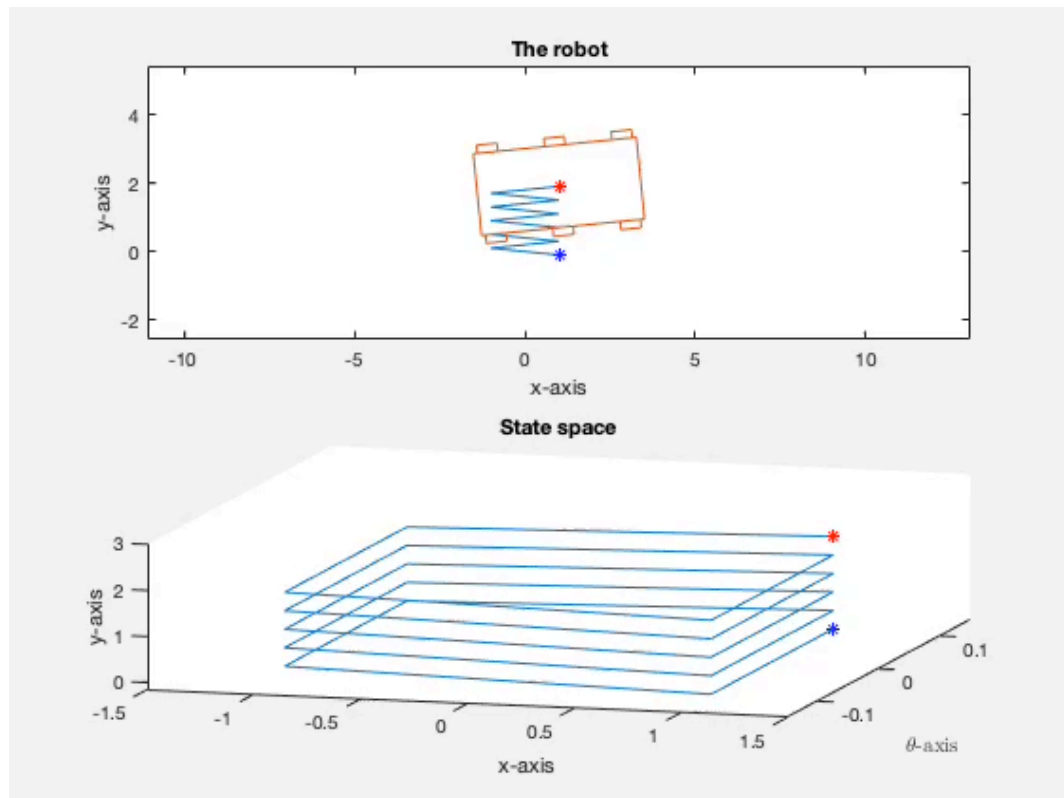
$$f(s) \in \mathbb{R}^n, \quad s \in [s_0, s_f]$$

- **Trajectory:** Is a path with a **time-parameterization** connected to it (i.e., also specifies the **velocity** or evolution of time along the path).

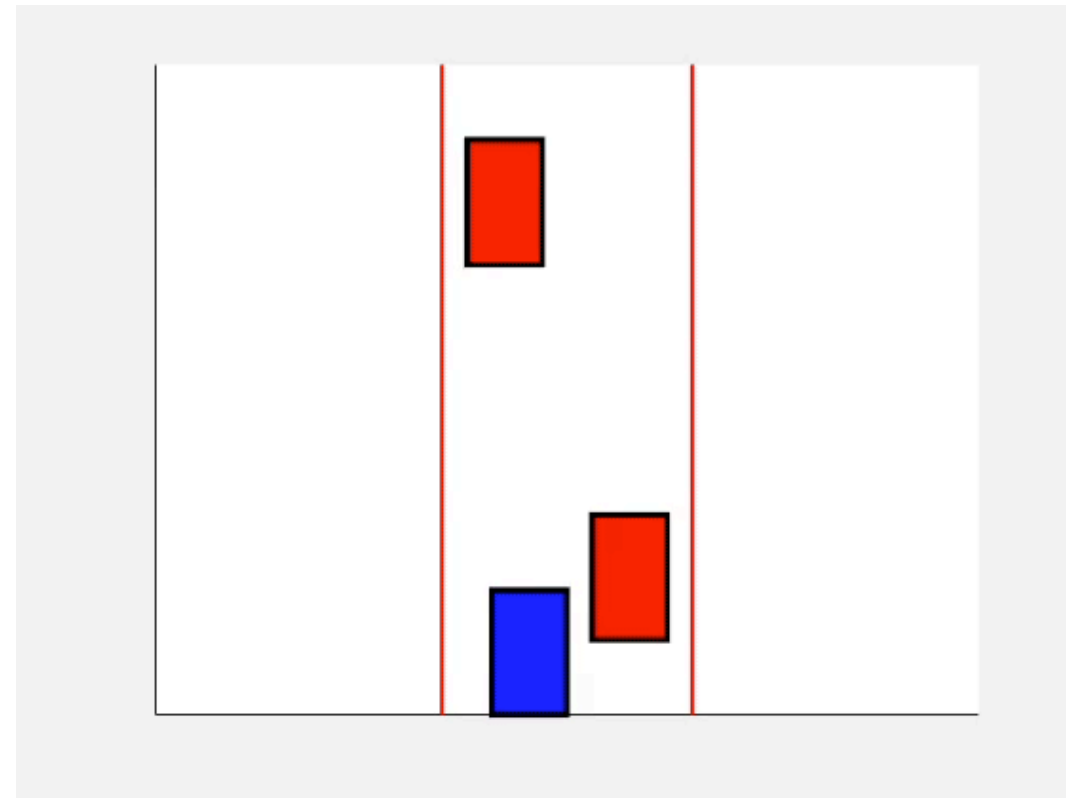
$$f(s(t)), \quad t \in [t_0, t_f]$$



Two examples



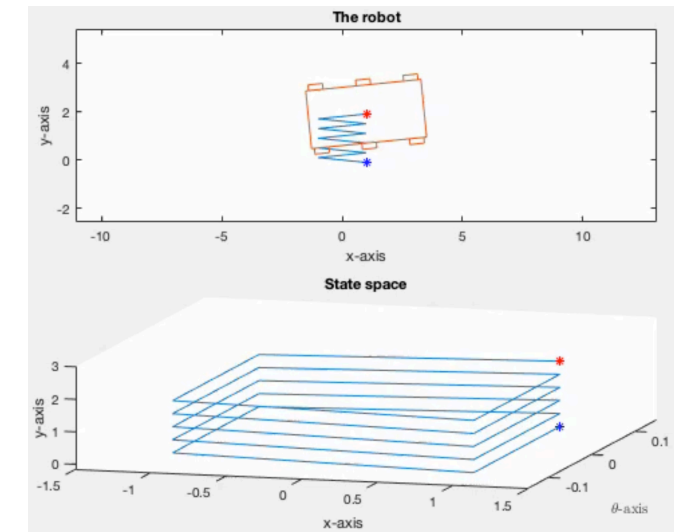
Path planning for a robot.
States: x , y , and θ



Trajectory planning for an overtaking scenario.
(The other cars are not moving in this case)
States: x , y , $\dot{x}$, $\dot{y}$

Motion Planning – Compare Lectures 2 & 3

- In Lecture 2, **discrete motion planning** was considered (example from Hand-in Exercise 1 to the right).
- In this lecture, the **differential motion constraints** are also considered.
- Recall the motion needed to make **parallel transitions** with a car from Lecture 3 (*cf.* **parallel parking**).



Differential Motion Constraints

- As seen in Lecture 3 on modeling of autonomous vehicles, many systems can be described by **differential equations** on the format:

$$\dot{x} = g(x, u)$$

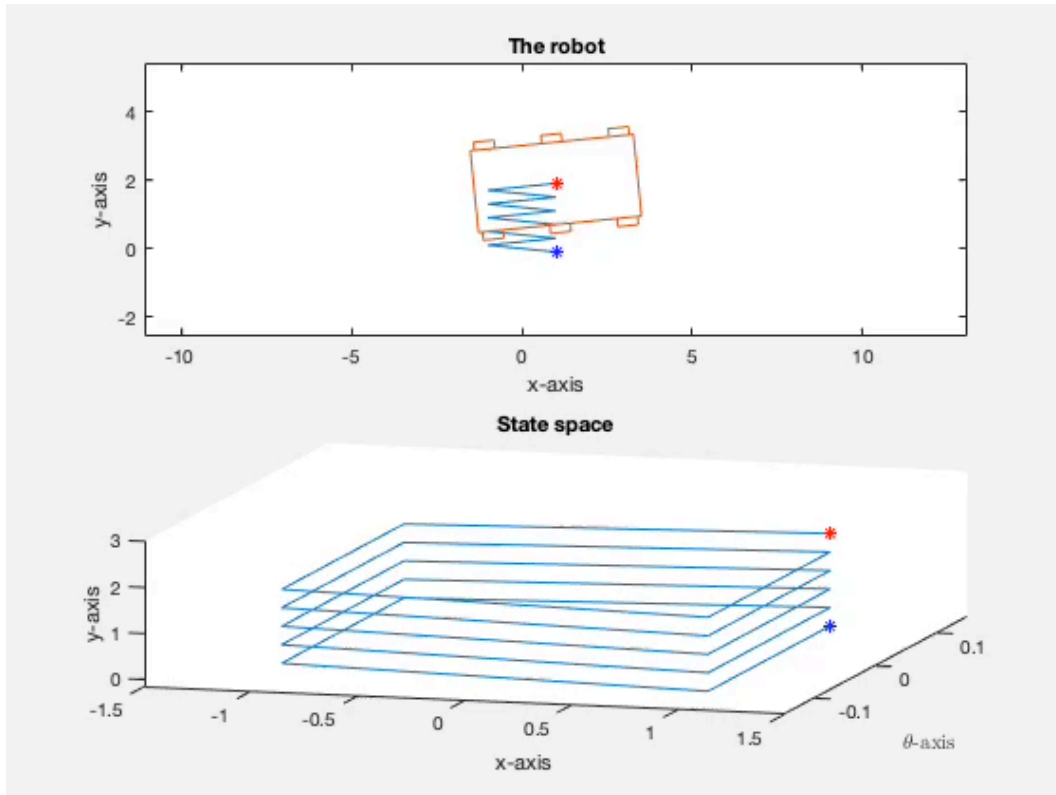
x – states
 u – inputs

- An essential task in **high-performance motion planning** is to take this constraint into account to make the motion plan feasible.

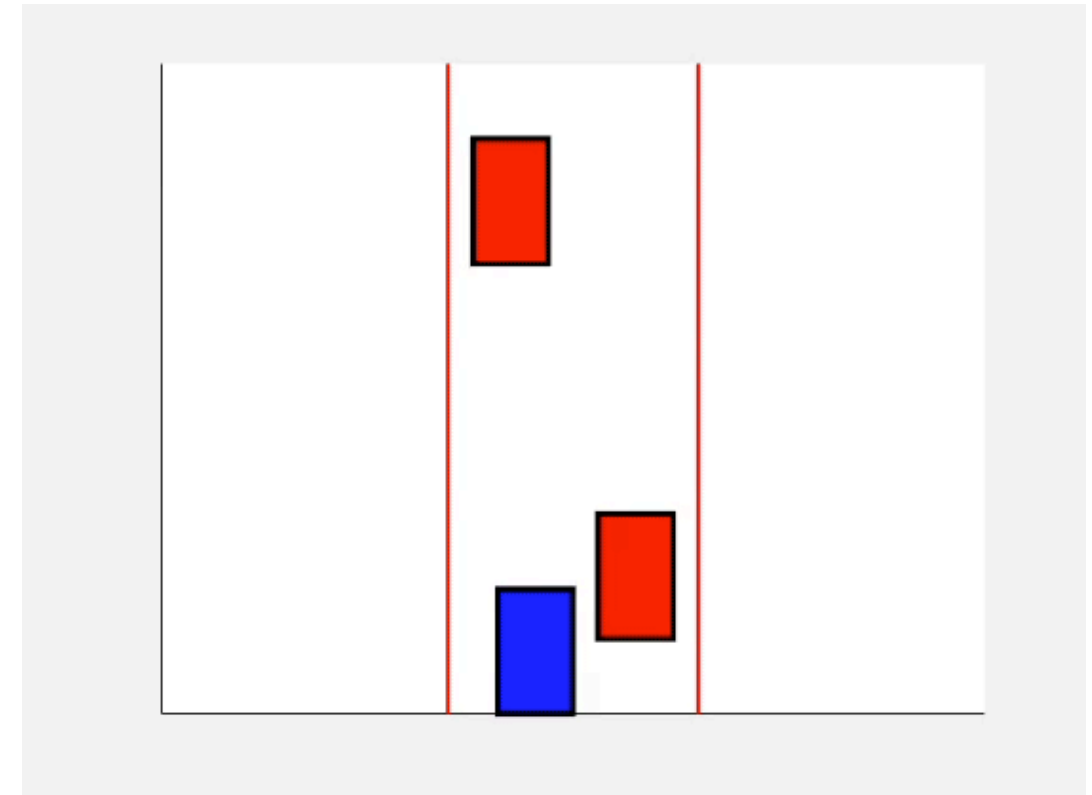
The Configuration Space

- The manifold defining the set of possible transformations of a vehicle from one state to another is called the **configuration space** $\mathcal{C}$
- The search for a **feasible motion plan** is done in the configuration space $\mathcal{C}$

Two examples



The configuration space has three dimensions (x, y, θ)



The configuration space has two dimensions (x, y)

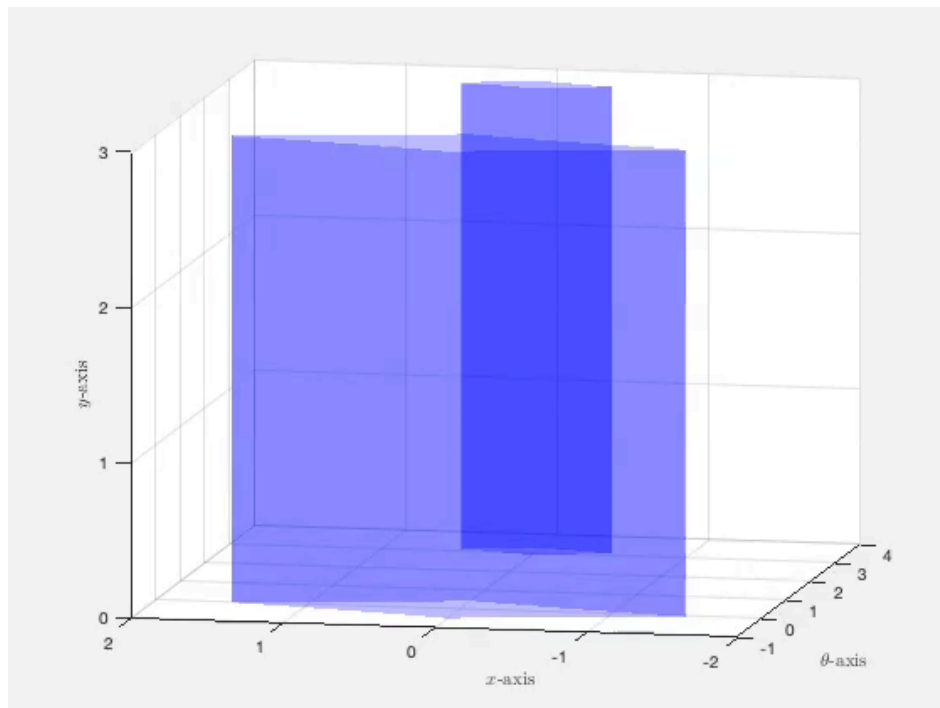
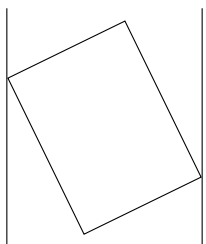
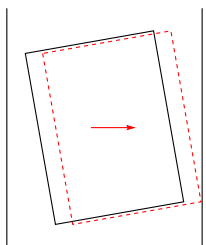
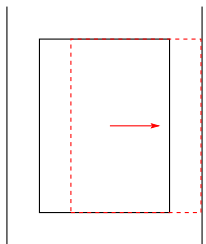
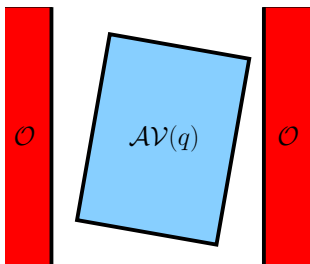
The Obstacle Region and the Free Space

- Assume a **world** $\mathcal{W}$ and an **obstacle region** $\mathcal{O}$. Further, let the **configuration** of the autonomous vehicle $\mathcal{AV}$ be defined by q in the configuration space $\mathcal{C}$. The **obstacle region** $\mathcal{C}_{obs}$ can then be defined as:

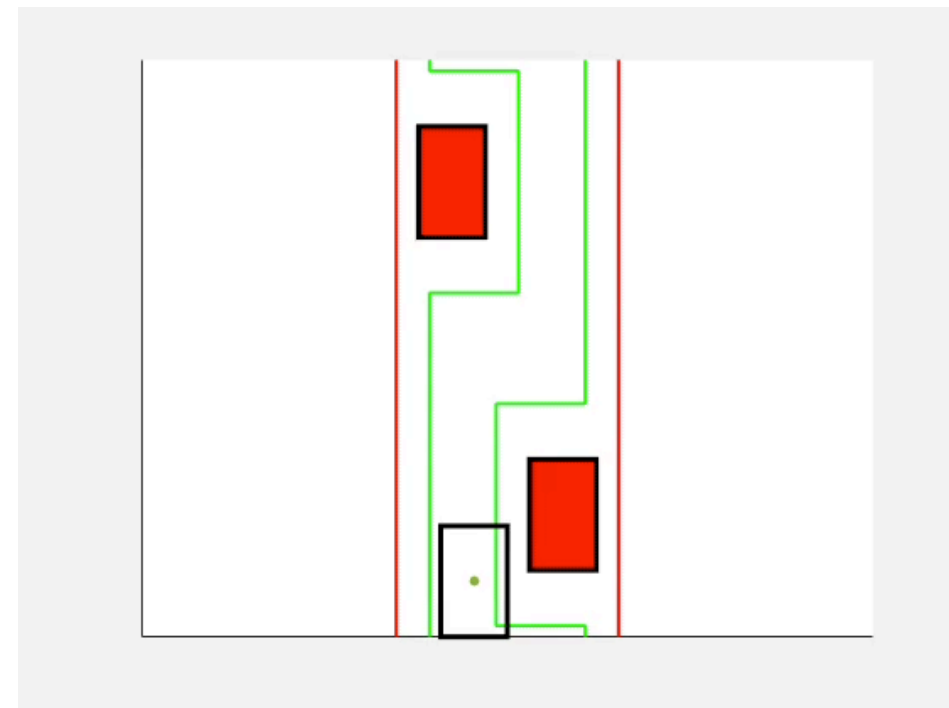
$$\mathcal{C}_{obs} = \{q \in \mathcal{C} \mid \mathcal{AV}(q) \cap \mathcal{O} \neq \emptyset\}$$

- The **free space** is accordingly defined as $\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{C}_{obs}$

Two examples



$\mathcal{C}_{\text{free}}$ is the region inside the two tubes

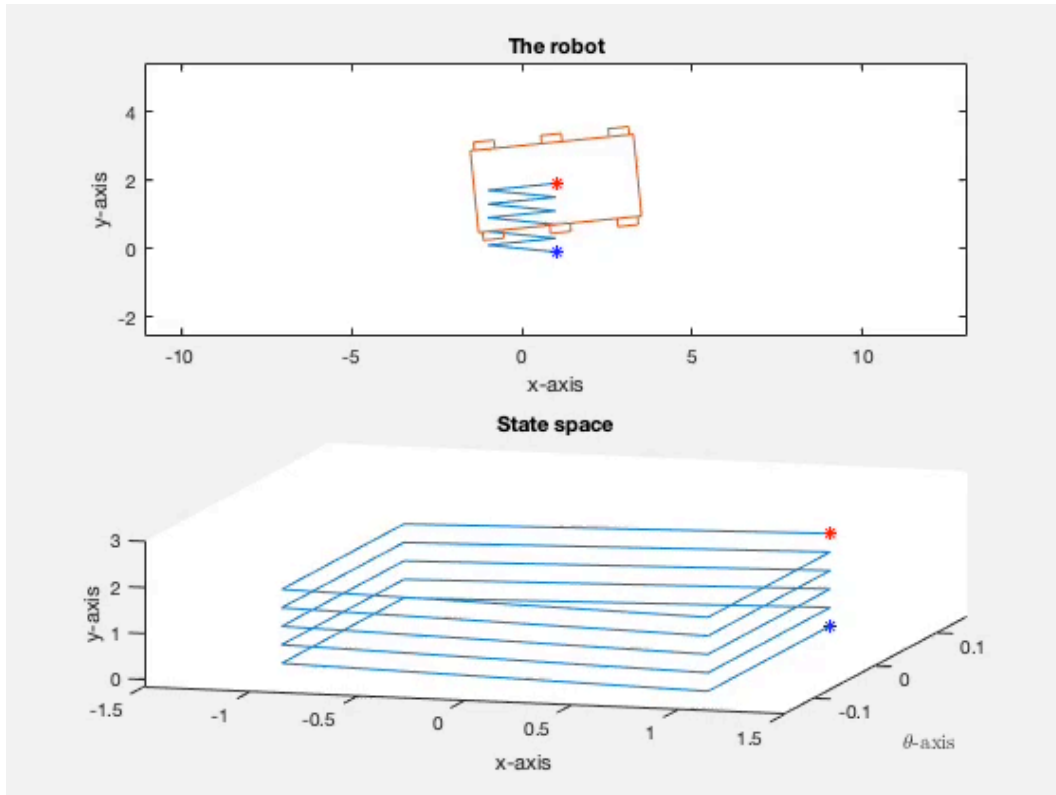


$\mathcal{C}_{\text{free}}$ is the region between the green lines

The State Space

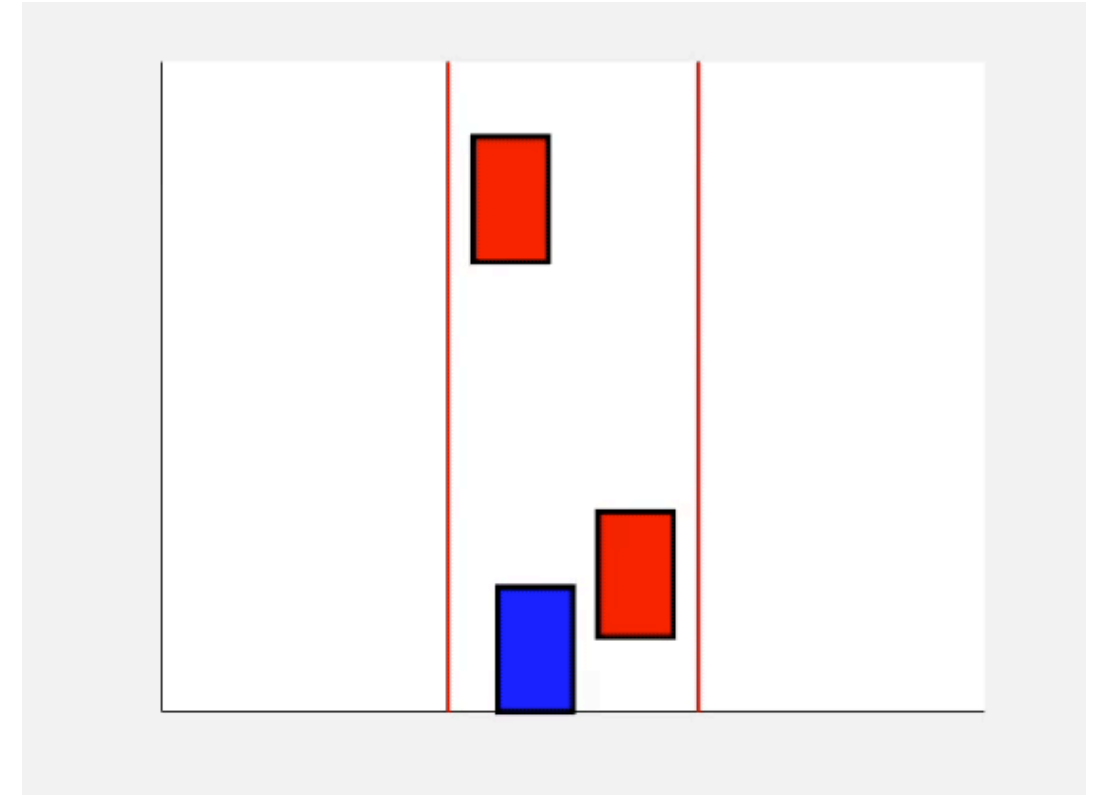
- The **configuration space** considers only the configuration q of the system.
- To consider also **higher-order derivatives** required for differential constraints (e.g., velocity and acceleration), the **phase space** is used.
- The phase space $\mathcal{X}$ has **higher dimension** than the configuration space (typically both position and velocity variables).
- In **motion planning**, the state space is the configuration space or the phase space, depending on the planning task.

Two examples



The state space is equal to the configuration space.

States: x , y , and θ



The state space is equal to the phase space.

States: x , y , $\dot{x}$, $\dot{y}$

Example: The State Space for a Double Integrator

- The differential equation for a **double integrator** with the input u is

$$\ddot{q} = u$$

- With the states $x = (x_1, x_2) = (q, \dot{q})$ and the state space $\mathcal{X} = \mathbb{R}^2$ the **state-space dynamic equations** for the double integrator can be written as

$$\begin{aligned} \dot{x}_1 &= x_2, \\ \dot{x}_2 &= u \end{aligned}$$
$$x = (q, \dot{q})$$

Obstacle Region in the Phase Space

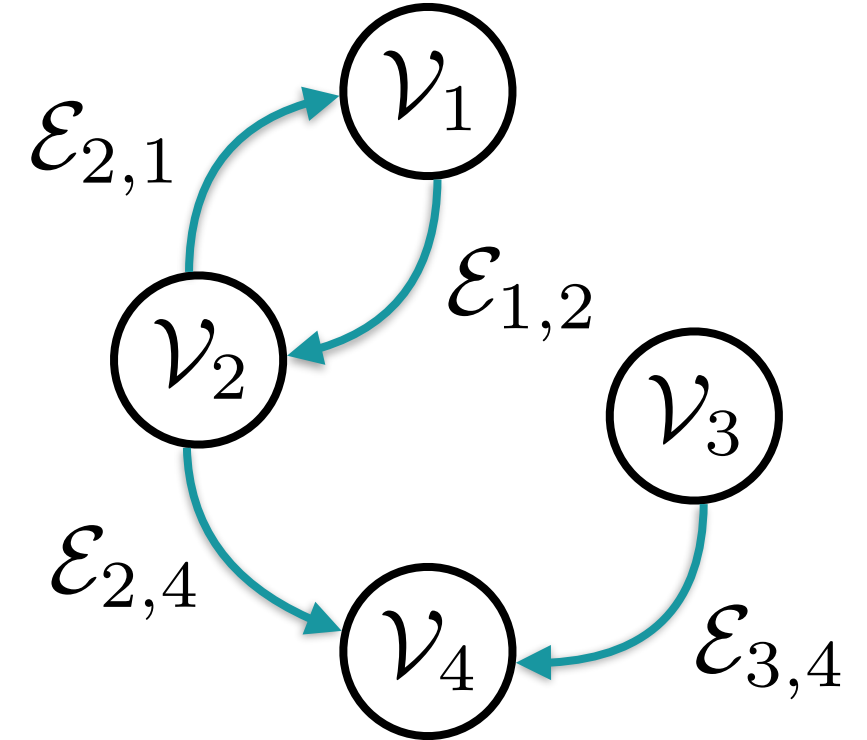
- The **obstacle region** in the phase space is defined as

$$\mathcal{X}_{\text{obs}} = \{x \in \mathcal{X} \mid \Xi(x) \in \mathcal{C}_{\text{obs}}\}$$

- The mapping Ξ gives the **configuration variables** from the complete set of states and $\mathcal{X}_{\text{free}} = \mathcal{X} \setminus \mathcal{X}_{\text{obs}}$.
- If the state space is the configuration space, then it **coincides** with $\mathcal{C}_{\text{obs}}$

Directed Graphs for Motion Planning

- A **directed graph** $\mathcal{G}$, comprising **vertices** $\mathcal{V}$ and **edges** $\mathcal{E}$.
- The direction of an edge between two vertices indicates the **transition direction**.
- A **vertex** is also commonly known as a **node**.



The Path-Planning Problem

- Let q_I be the **initial configuration** and q_G the **desired goal configuration**.
- The **path-planning problem** consists of computing a path f for the autonomous vehicle $\mathcal{AV}$ according to

$$f : [0, 1] \rightarrow \mathcal{C}_{\text{free}}, \text{ with } f(0) = q_I, \ f(1) = q_G$$

- The path must **only reside in** $\mathcal{C}_{\text{free}}$.

The Motion-Planning Problem w. Differential Constraints

- Let x_I be the **initial state** and x_G the desired **goal state**.
- The motion-planning problem **with differential constraints** is to compute the control inputs $u(t)$ for the vehicle $\mathcal{AV}$ such that the corresponding state trajectory $x(t)$ satisfies

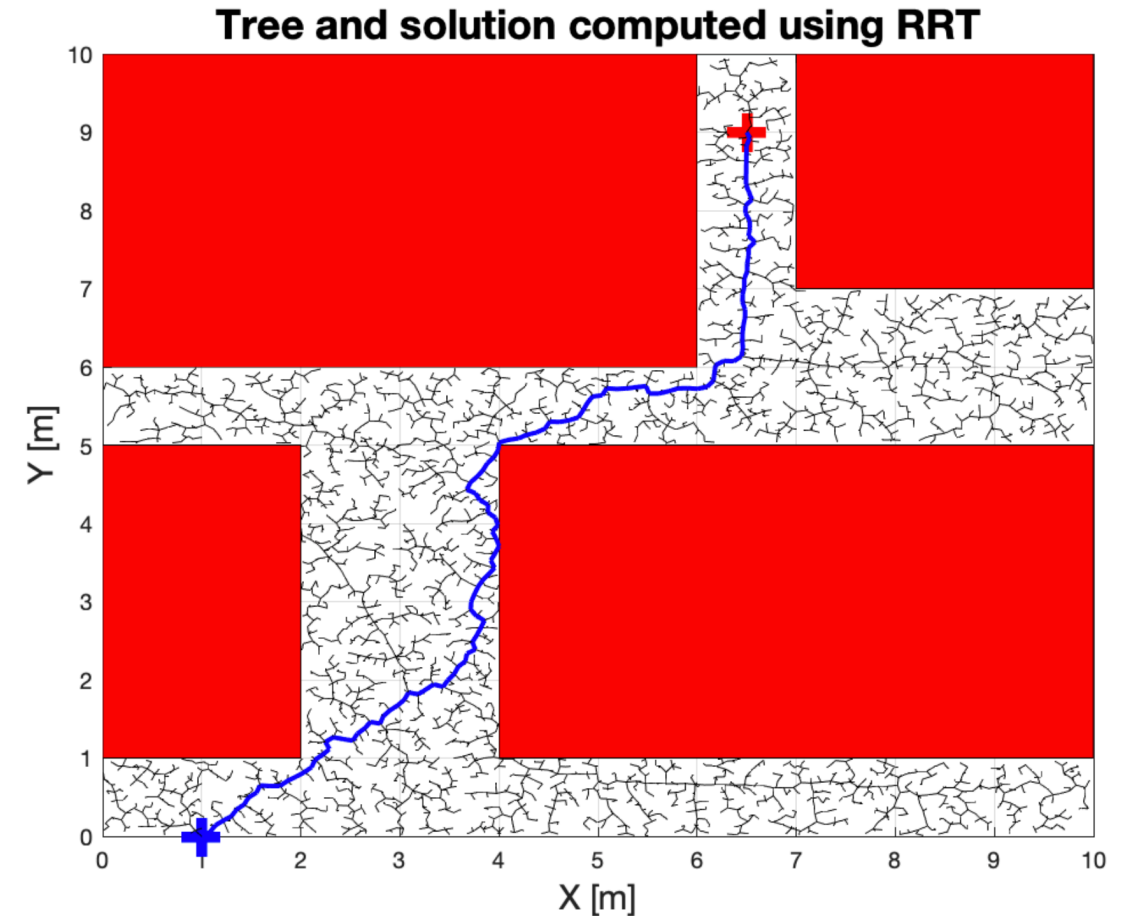
$$x(t) \in \mathcal{X}_{\text{free}}, \quad x(0) = x_I, \quad x(T) = x_G$$

- Often **constraints** on $u(t)$ and other **specifications** on $x(t)$.

Rapidly-Exploring Random Trees (RRTs) for Motion Planning

The Basic Idea of RRTs (1/2)

- The basic idea behind **rapidly-exploring random trees** (RRTs) is to **incrementally construct** a tree in the state space by **sampling**.
- A **tree** is a graph where any two vertices are connected by **exactly one path** (each **vertex** corresponds to a **state** and the **edge** is the **feasible motion segment** between two vertices).



The Basic Idea of RRTs (2/2)

- The **sampling** can be **random** or any other strategy leading to a **dense sequence of samples**.
- RRT computes a **feasible path**, whereas **RRT*** also aims to **minimize a cost** associated with the motion.

Sampling in the State Space

- Construction of the tree relies on **efficient sampling** of the state space ($\mathcal{C}_{\text{free}}$ or $\mathcal{X}_{\text{free}}$).
- **Sequence of the samples** important, not only the set of samples.
- Sampling can be both **random** and **deterministic**.
- Special attention required when sampling in **orientation spaces** such as the 3D rotation group $\text{SO}(3)$.

Basic Version of RRT (1/2)

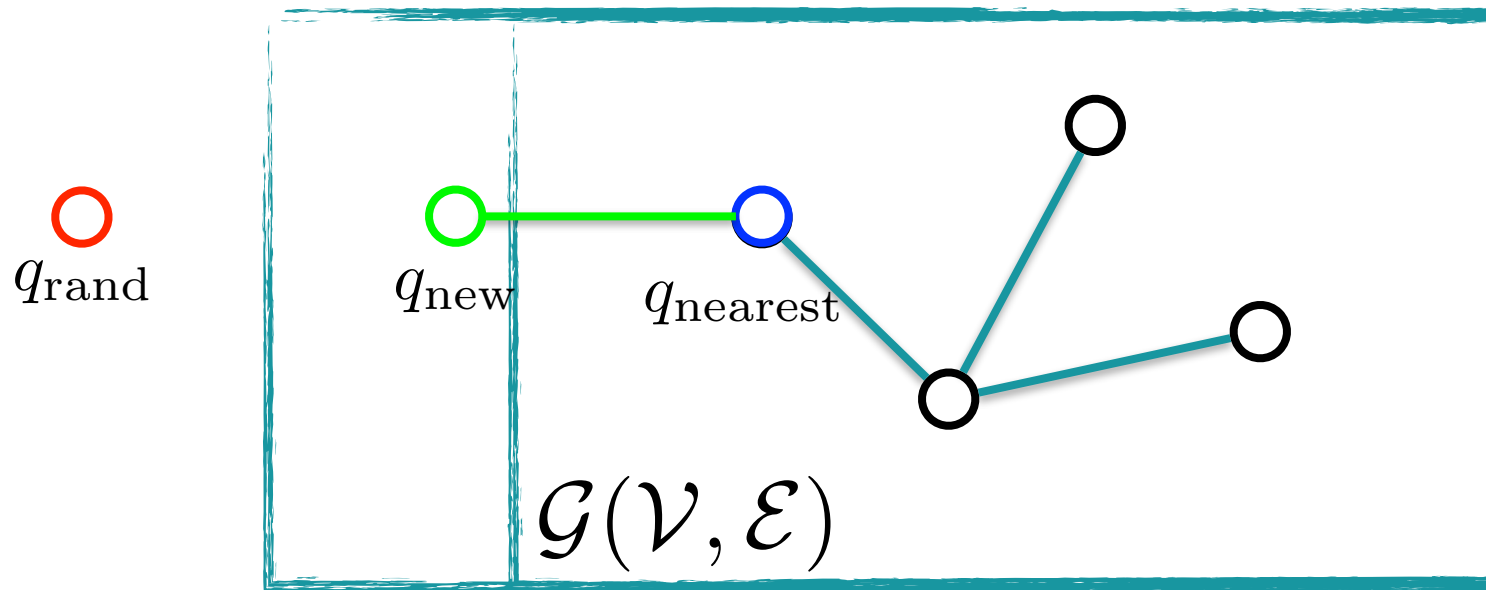
Algorithm 1: RRT w/o. Differential Constraints

```

1   $\mathcal{V} \leftarrow \{q_I\}, \mathcal{E} \leftarrow \emptyset;$ 
2  for  $i = 1, \dots, N$  do:
3       $q_{\text{rand}} \leftarrow \text{Sample};$ 
4       $q_{\text{nearest}} \leftarrow \text{Nearest}(\mathcal{G} = (\mathcal{V}, \mathcal{E}), q_{\text{rand}});$ 
5       $q_{\text{new}} \leftarrow \text{Steer}(q_{\text{nearest}}, q_{\text{rand}});$ 
6      if  $\text{ObstacleFree}(q_{\text{nearest}}, q_{\text{new}})$  then
7           $\mathcal{V} \leftarrow \mathcal{V} \cup \{q_{\text{new}}\};$ 
8           $\mathcal{E} \leftarrow \mathcal{E} \cup \{(q_{\text{nearest}}, q_{\text{new}})\};$ 
9  return  $\mathcal{G} = (\mathcal{V}, \mathcal{E});$ 

```

RRT – Example w/o. Obstacles



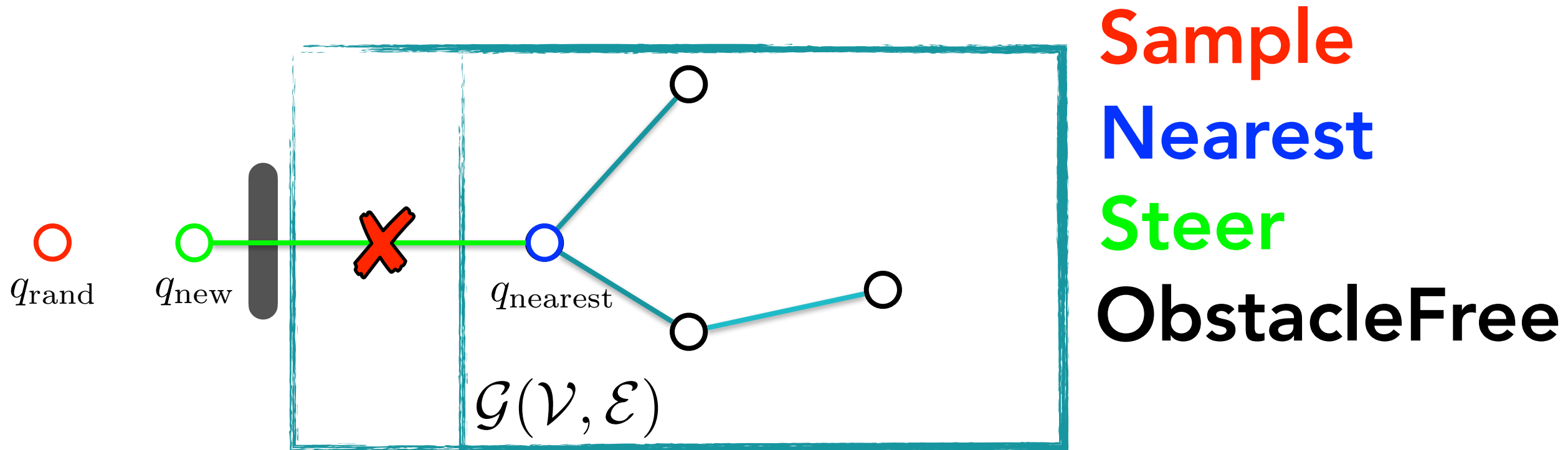
Sample

Nearest

Steer

ObstacleFree

RRT – Example w. Obstacle



Basic Version of RRT (2/2)

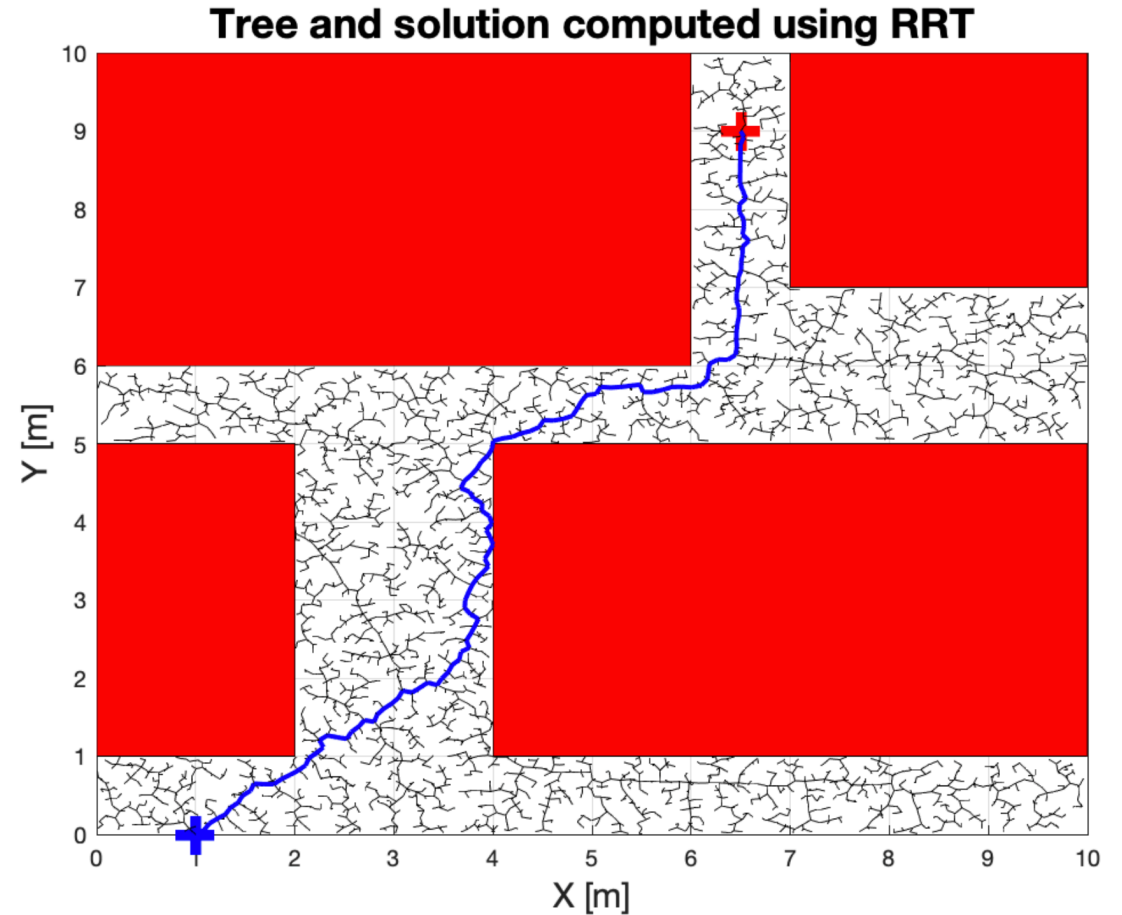
- **Sample:** Gives a sample in the free state space.
- **Nearest:** Provides the vertex in the tree that is closest to the sampled state.
- **Steer:** In general this is a so called two-point boundary value problem (TPBV). Construct a path from the nearest vertex towards the sampled state, often with a maximum step length (alternative strategies exist).
- **ObstacleFree:** Checks whether the path from the closest vertex in the graph to the new state is collision free.

RRT with Biased Sampling

- To speed up the process of reaching the **goal state**, a **bias** towards it can be introduced in the sampling.
- In practice: with a certain **pre-defined probability** β the sampled state is the goal state.
- In general, the sampling could be biased to be, e.g., in the **direction of the road** or **directions implied by the environment geometry**.

RRT – Example

- Plan a **path** from the initial state (blue cross) to the goal state (red cross) using **RRT**.
- The red boxes are **obstacles**.
- A particle moving in a 2D world.
- How to handle **differential motion constraints**?



RRT

RRT with Differential Motion Constraints (1/2)

Algorithm 2: RRT w. Differential Constraints

```

1   $\mathcal{V} \leftarrow \{x_I\}, \mathcal{E} \leftarrow \emptyset;$ 
2  for  $i = 1, \dots, N$  do:
3       $x_{\text{rand}} \leftarrow \text{Sample};$ 
4       $x_{\text{nearest}} \leftarrow \text{Nearest}(\mathcal{G} = (\mathcal{V}, \mathcal{E}), x_{\text{rand}});$ 
5       $(x_{\text{new}}, \tilde{u}^p) \leftarrow \text{Steer}(x_{\text{nearest}}, x_{\text{rand}});$ 
6      if  $\text{ObstacleFree}(x_{\text{nearest}}, x_{\text{new}})$  then
7           $\mathcal{V} \leftarrow \mathcal{V} \cup \{x_{\text{new}}\};$ 
8           $\mathcal{E} \leftarrow \mathcal{E} \cup \{(x_{\text{nearest}}, x_{\text{new}}; \tilde{u}^p)\};$ 
9  return  $\mathcal{G} = (\mathcal{V}, \mathcal{E});$ 

```


RRT with Differential Motion Constraints (2/2)

- **Sample:** Gives a sample in the free state space.
- **Nearest:** Provides the vertex in the tree that is closest to the sampled state.
- **Steer:** Apply the input $\tilde{u}^p$ from a pre-defined discrete set of possible control input trajectories, leading as close to the sampled state as possible.
 - The input trajectories are often of the same time duration.
 - A simulation of the motion equations can be used to compute the trajectories obtained with the different inputs.
- **ObstacleFree:** Checks whether the path from the closest vertex to the new state is collision free.

Introduction to RRT*

- **RRT*** is an extension of RRT targeting **optimal sampling-based motion planning**.
- Compared to RRT, **more steps** are performed **each time a new vertex** is added to the tree.
- In this lecture, **RRT* without differential motion constraints** are considered.
- **Extensions** covering differential motion constraints exist, but are not covered in the lectures, however, a possible mini-project.

RRT* – Optimal Motion Planning (1/2)

Algorithm 3: RRT* w/o. Differential Constraints

```

1   $\mathcal{V} \leftarrow \{q_I\}, \mathcal{E} \leftarrow \emptyset;$ 
2  for  $i = 1, \dots, N$  do:
3       $q_{\text{rand}} \leftarrow \text{Sample};$ 
4       $q_{\text{nearest}} \leftarrow \text{Nearest}(\mathcal{G} = (\mathcal{V}, \mathcal{E}), q_{\text{rand}});$ 
5       $q_{\text{new}} \leftarrow \text{Steer}(q_{\text{nearest}}, q_{\text{rand}});$ 
6      if  $\text{ObstacleFree}(q_{\text{nearest}}, q_{\text{new}})$  then
7           $Q_{\text{near}} \leftarrow \text{Near}(\mathcal{G} = (\mathcal{V}, \mathcal{E}), q_{\text{new}}, \gamma);$ 
8           $\mathcal{V} \leftarrow \mathcal{V} \cup \{q_{\text{new}}\};$ 
9           $q_{\text{min}} \leftarrow q_{\text{nearest}}; c_{\text{min}} \leftarrow \text{Cost}(q_{\text{nearest}}) + \text{CostL}(\text{Line}(q_{\text{nearest}}, q_{\text{new}}));$ 
10         foreach  $q_{\text{near}} \in Q_{\text{near}}$  do
11             if  $\text{CollisionFree}(q_{\text{near}}, q_{\text{new}}) \ \&\&$ 
12                  $\text{Cost}(q_{\text{near}}) + \text{CostL}(\text{Line}(q_{\text{near}}, q_{\text{new}})) < c_{\text{min}}$  then
13                      $q_{\text{min}} \leftarrow q_{\text{near}}; c_{\text{min}} \leftarrow \text{Cost}(q_{\text{near}}) + \text{CostL}(\text{Line}(q_{\text{near}}, q_{\text{new}}));$ 
14              $\mathcal{E} \leftarrow \mathcal{E} \cup \{(q_{\text{min}}, q_{\text{new}})\};$ 
15         foreach  $q_{\text{near}} \in Q_{\text{near}}$  do
16             if  $\text{CollisionFree}(q_{\text{new}}, q_{\text{near}}) \ \&\&$ 
17                  $\text{Cost}(q_{\text{new}}) + \text{CostL}(\text{Line}(q_{\text{new}}, q_{\text{near}})) < \text{Cost}(q_{\text{near}})$  then
18                      $q_{\text{parent}} \leftarrow \text{Parent}(q_{\text{near}});$ 
19                      $\mathcal{E} \leftarrow (\mathcal{E} \setminus \{(q_{\text{parent}}, q_{\text{near}})\}) \cup \{(q_{\text{new}}, q_{\text{near}})\};$ 
20 return  $\mathcal{G} = (\mathcal{V}, \mathcal{E});$ 

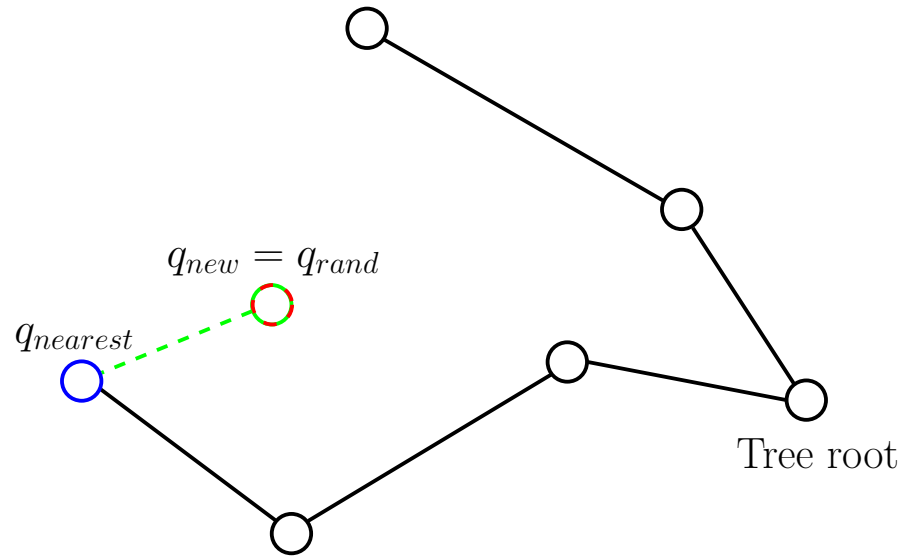
```

Use path with minimum cost for connecting the new state to the tree.

Re-wire the vertices in a neighborhood of the new state that can reduce the cost.

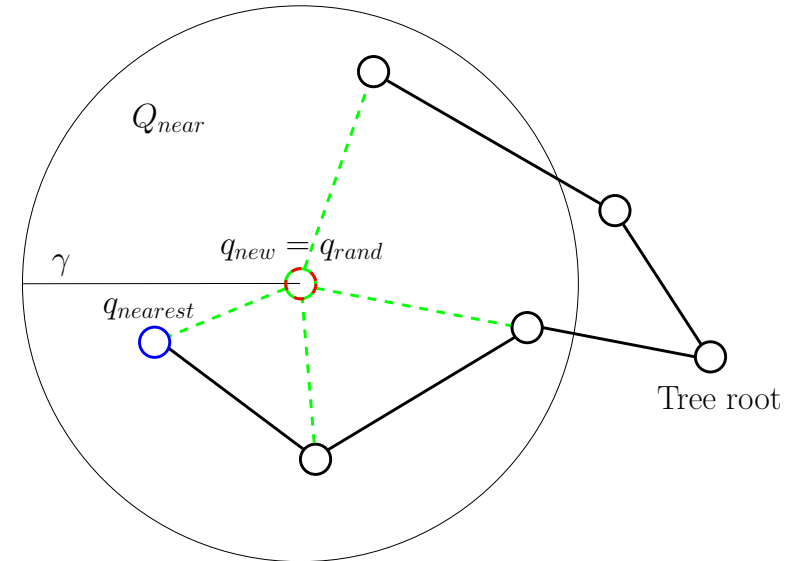
RRT* example: Minimize distance

First steps are the same as before



The node q_{new} is added to the set of nodes $\mathcal{V}$, and the edge between q_{new} and $q_{nearest}$ is considered as a first candidate to connect q_{new} to the tree $\mathcal{G} = (\mathcal{V}, \mathcal{E})$.

Can you see any possible improvements?

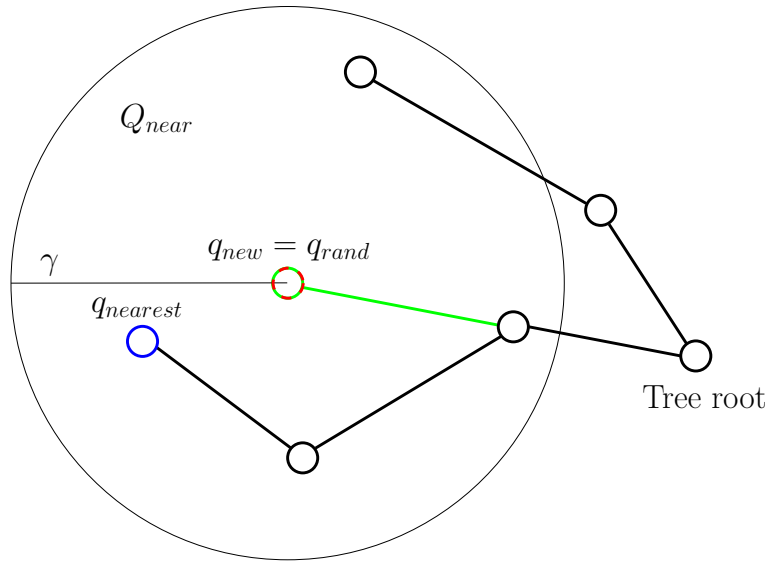


Define the set Q_{near} as the set of all nodes within a circle with radius γ and center q_{new} . Choose the node in Q_{near} which gives the shortest path from q_{new} to the tree root.

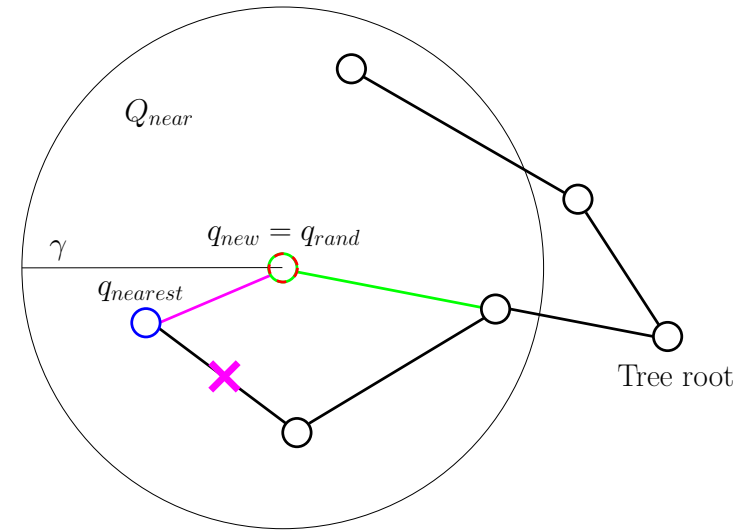
RRT* example: Minimize distance

41

The green edge is now added to the set of edges $\mathcal{E}$



Can you see any further improvements considering the nodes in Q_{near} ?



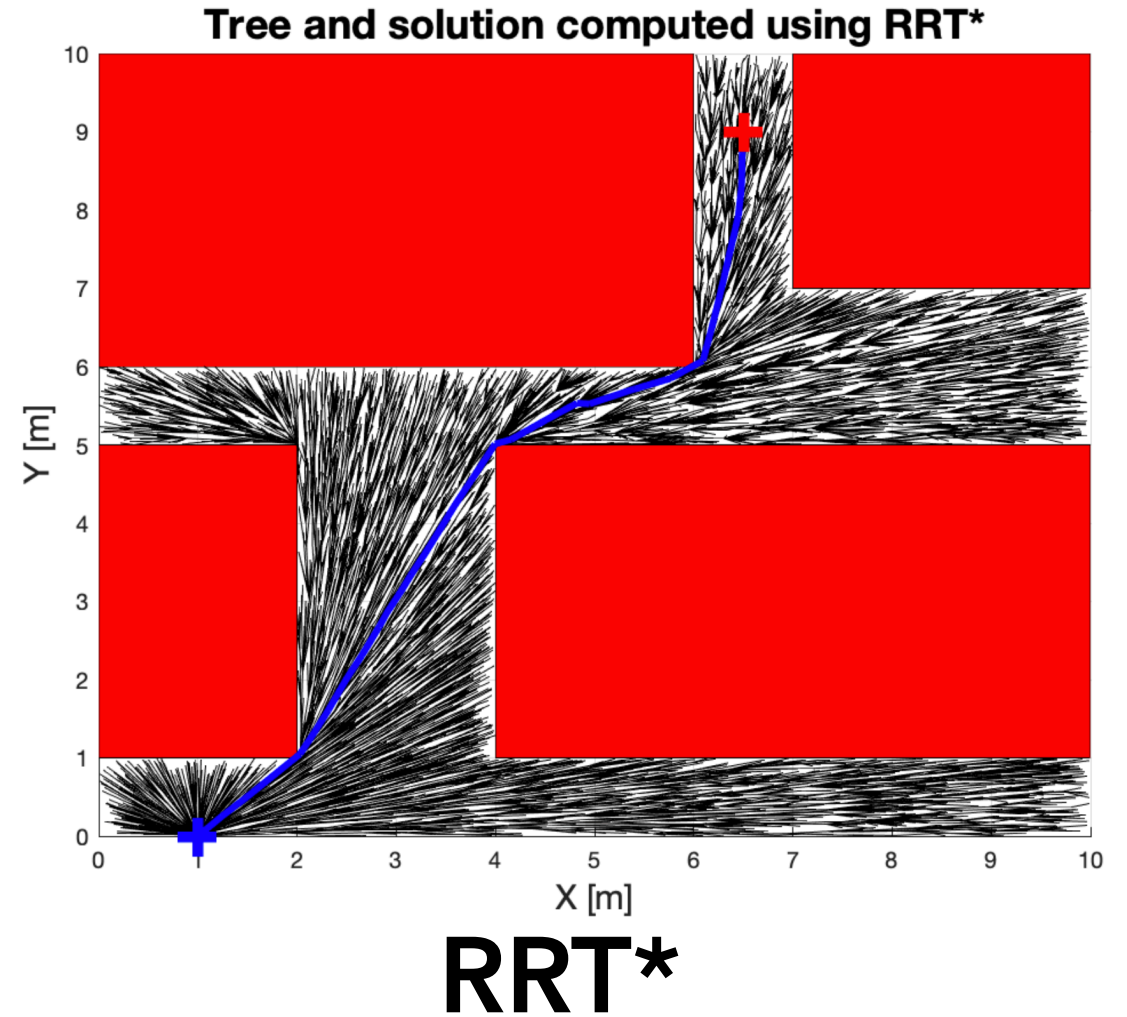
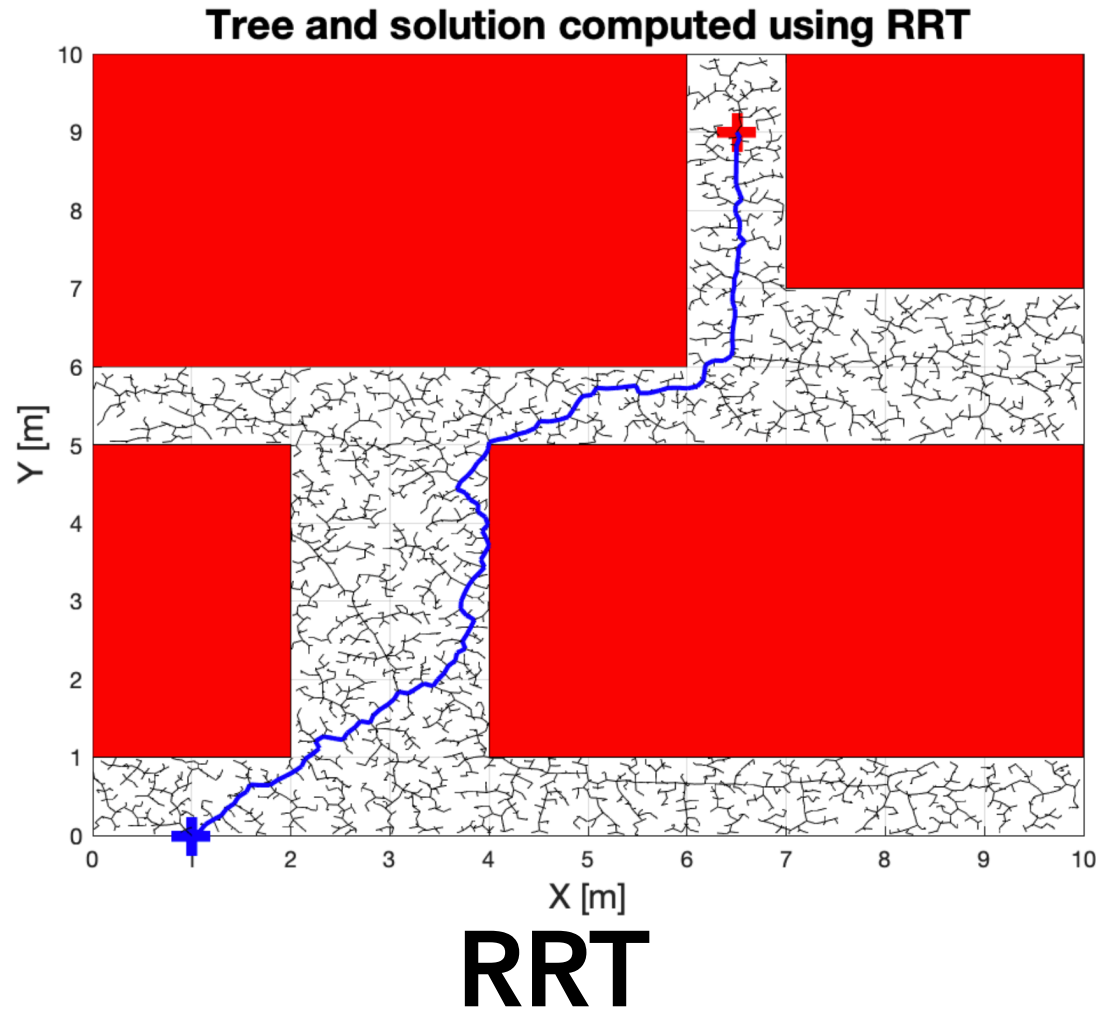
Consider all nodes in Q_{near} and check if the distance to the tree root becomes smaller by connecting it via the new node q_{new} . If this is the case, rewire the node by adding the edge to q_{new} and remove the edge to parent node.

RRT* – Optimal Motion Planning (2/2)

- Same functions as in the RRT algorithm, but with certain additional functions:
 - **Near**: Provides all vertices within a ball with radius γ in the tree.
 - **Cost**: The cost-to-come to the current vertex from the tree root (initial state).
 - **CostL**: The cost of moving along a straight line from the specified vertices.
 - **Line**: Gives the path between two vertices as a straight line.
 - **Parent**: Gives the vertex that is parent to the current vertex in the tree.
- Note that in RRT* **more steps are done** each time a **new state** is obtained:
 - Make sure that the new vertex is connected along the path with **minimum cost**.
 - **Re-wire** the existing tree so as to try to **minimize the cost** gradually.

Comparison of RRT and RRT*

43



RRT – Implementation Aspects

- Choice of norm for **measure of closeness** between vertices.
- **Bi-directional growth** of two RRTs (or even more) simultaneously.

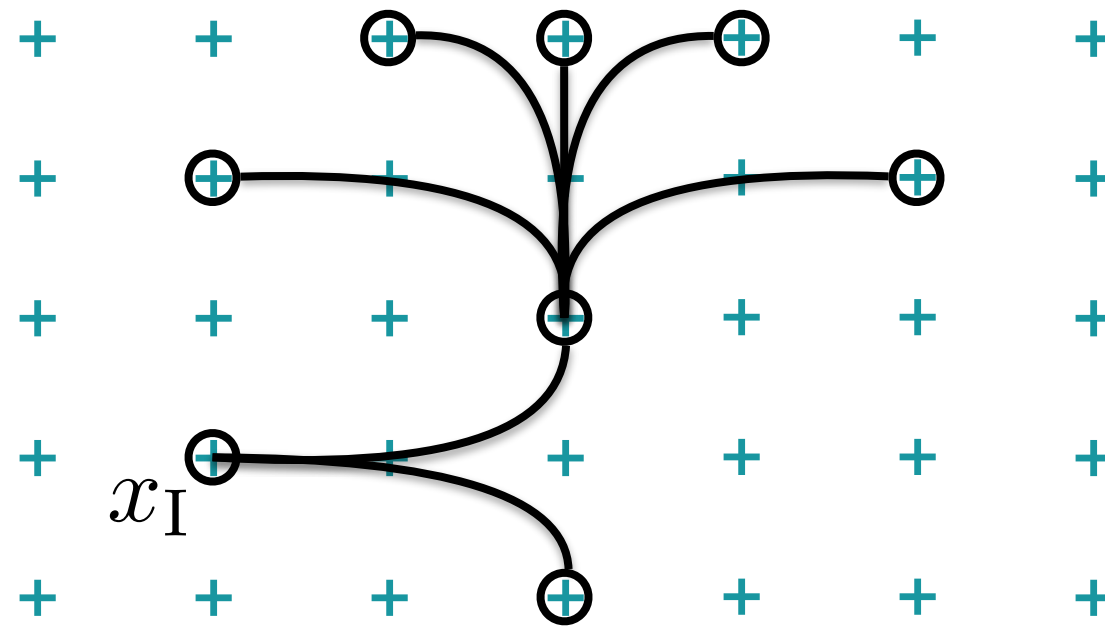
Planning Using Motion Primitives and State Lattices

General Idea with Lattice Planning (1/2)

- In contrast to the discrete graph-search problems in Lecture 2, a **motion-planning problem** with a **continuous-time differential motion constraint** is an infinite-dimensional problem.
- No graph with a **finite set of nodes** exists.
- Idea: Select a **finite number of control-signal segments** (***motion primitives***) and choose them such that the start and end states of their corresponding trajectories are on a **discrete lattice** (i.e., a regular grid)
⇒ Back on the graph-search problem from Lecture 2.

General Idea with Lattice Planning (2/2)

- The solution is a sequence of motion primitives.

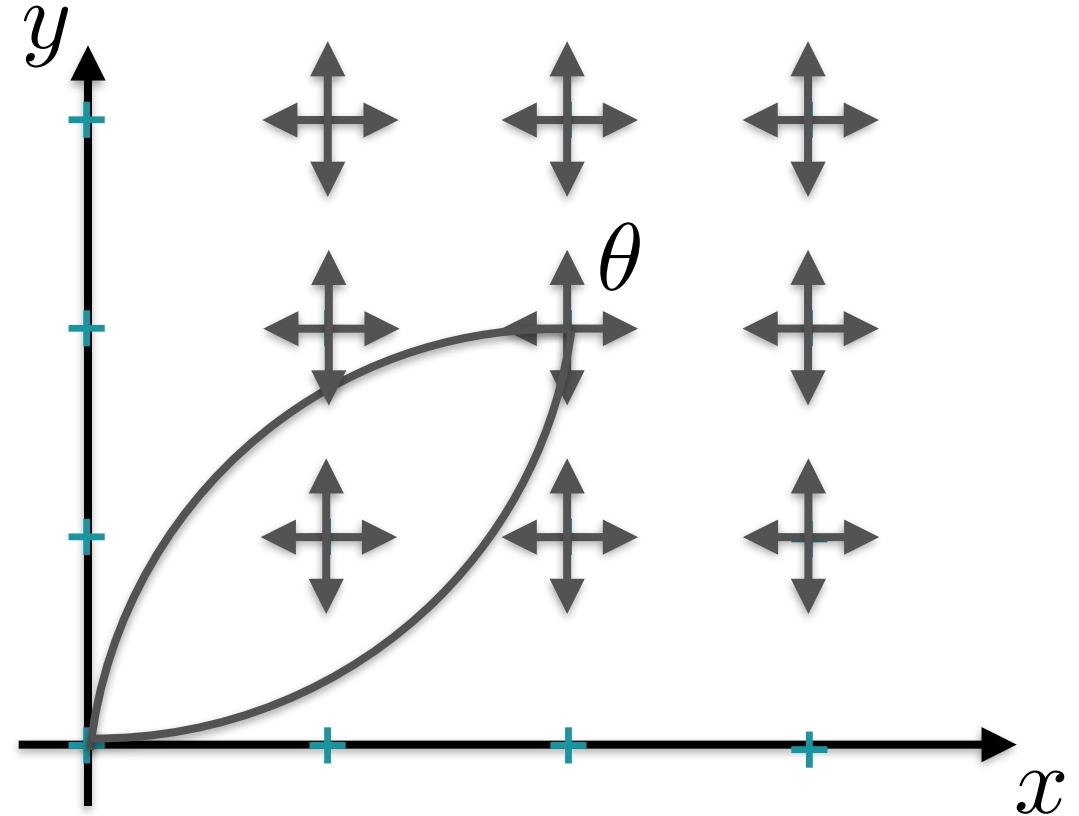


State Lattice – Discretization of State Space

- To transform the original continuous motion-planning problem to a discrete (finite-dimensional) problem, the state space is **discretized** into the set $\mathcal{X}_d$.
- All state variables are discretized, e.g., using a uniform grid, which results in a so called **state lattice**.
- The planning is then relying on using a **sequence of motion segments**, each bringing the vehicle from one state in the discrete set $\mathcal{X}_d$ to another (never outside this set).

State Lattice – Example

- An example where the translational coordinates (x, y) are discretized **uniformly** and four different orientations θ are allowed at each positional state.
- The arrows indicate the set of **allowed orientations**.



Motion Primitives

- The **transition** from one state in $\mathcal{X}_d$ to the next must obey the motion equations of the system (i.e., a differential constraint).
- To encode a (discrete) set of **possible state transitions** within the lattice, a set of **motion primitives** $\mathcal{MP}$ is computed.
- Each motion primitive must start and end at one state in the set $\mathcal{X}_d$ (though not necessary that all states are connected).

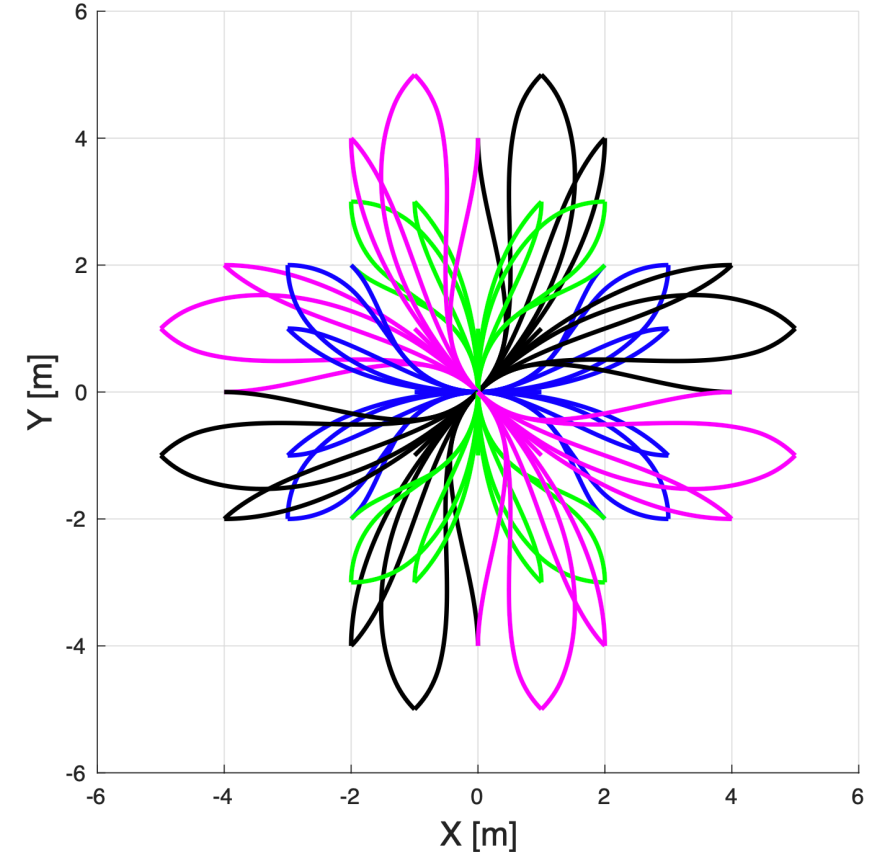
Motion Primitives – Examples

- Consider a **vehicle** with dynamics

$$\begin{aligned}\dot{x} &= v \cos(\theta), & x, y &- \text{translational coordinates} \\ \dot{y} &= v \sin(\theta), & \theta &- \text{orientation coordinate} \\ \dot{\theta} &= v/L \tan(u) & v &- \text{velocity} \\ & & u &- \text{steering angle (input)} \\ & & L &- \text{wheelbase}\end{aligned}$$
- A **sample set of motion primitives** (constant velocity) with the discretized orientation angles

$$\Theta_d = \{0, \pi/4, \pi/2, 3\pi/4, \pi, -3\pi/4, -\pi/2, -\pi/4\}$$

Motion Primitives for Pre-Defined State Lattice



Computation of Motion Primitives

- For certain types of vehicles, **analytical solutions** exist for a dynamically feasible motion between an initial state and another desired state (e.g., Dubins path, see Lecture 3).
- In other cases, **numerical optimal control** can be used to compute (optimal) motion segments to be used as motion primitives (see Lecture 5 and Hand-in Exercise 2).
- Typically a **cost** (e.g., path length, maneuver time, utilization of control inputs) is associated with each motion primitive.
- **Constraints on control inputs** introduced in the computations of the primitives.
- Utilize **symmetries** and translational **invariances** in the computations (translation, rotations, and mirroring).

Planning on the Lattice Using Motion Primitives (1/4)

- Recall the **graph-search methods** studied in Lecture 2.
- Using the (finite set of) motion primitives, the **motion-planning problem** can now be solved using graph search.
- In each state $x_d \in \mathcal{X}_d$ in the discretized state space, the **possible transitions** are given by

$$\tilde{u}^p \in \mathcal{MP}(x_d)$$

Planning on the Lattice Using Motion Primitives (2/4)

- After application of a possible motion primitive, a **collision check** gives whether the motion is feasible (given current obstacles), i.e., it must be verified that

$$\text{Path}(x_d, \tilde{u}^p) \in \mathcal{X}_{\text{free}}$$

- Graph-search methods such as **Dijkstra's** and **A*** applicable for the planning.
- The search is **terminated** when x_G has been selected for expansion.

Planning on the Lattice Using Motion Primitives (3/4)

- Recall the A* graph-search algorithm from Lecture 2.

Algorithm 4: A* with Motion Primitives

```

1   $C(x_I) = 0;$ 
2   $Q.insert(x_I, C(x_I) + h(x_I));$ 
3   $V(x_I) = x_I;$ 
4   $P(x_I) = \emptyset;$ 
5  while  $Q \neq \emptyset$  do
6       $x = Q.pop();$ 
7      if  $x = x_G$ 
8          return SUCCESS;
9      foreach  $\tilde{u}^p \in \mathcal{MP}(x)$  do
10          $x' = \text{NextState}(x, \tilde{u}^p);$ 
11         if  $\text{Path}(x, \tilde{u}^p) \notin \mathcal{X}_{\text{free}}$  then
12             continue;
13         if not  $V(x') \mid\mid C(x') > C(x) + \text{Cost}(x, x')$  then
14              $V(x') = x;$ 
15              $P(x') = \tilde{u}^p;$ 
16              $C(x') = C(x) + \text{Cost}(x, x');$ 
17              $Q.insert(x', C(x') + h(x'));$ 
18  return FAILURE;
  
```

Q – priority queue

$C(x)$ – cost-to-come to state x

$h(x)$ – heuristic cost-to-go to goal state for state x

$V(x)$ – parent node to state x

$P(x)$ – motion primitive used to reach state x

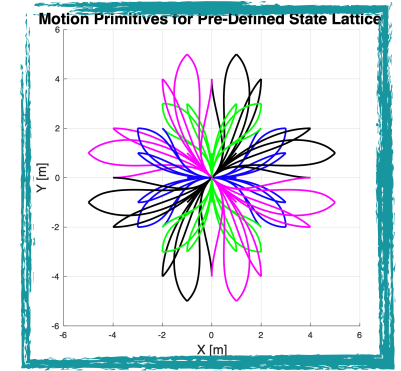
Planning on the Lattice Using Motion Primitives (4/4)

- Same functions as in the **A* graph-search algorithm** from Lecture 2, but with some additional ones:
 - **NextState**: Provides the state reached after application of the considered motion primitive.
 - **Path**: Gives the path between the state x and the next state by application of the specific motion primitive $\tilde{u}_p$.
 - **Cost**: The cost to move between two specified states.

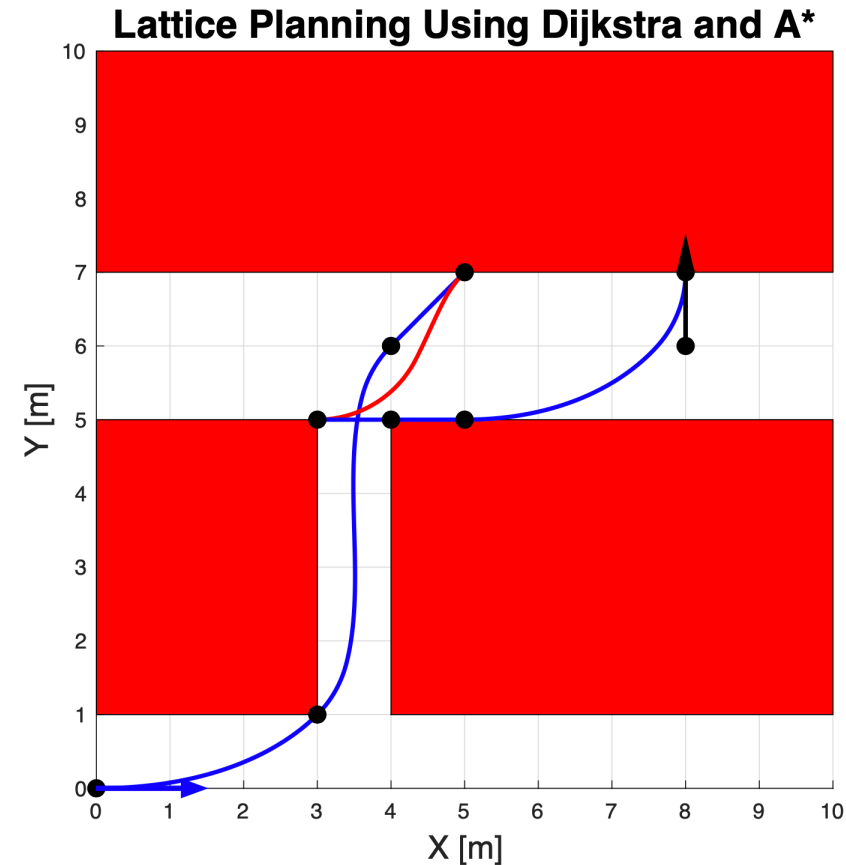
Lattice Planning – Implementation Aspects

- Choice of **state discretization non-trivial task**, closely connected to the specific vehicle dynamics and targeted planning situations.
- Cardinality of the sets $\mathcal{X}_d$ and $\mathcal{MP}$ influences the **online computational time**, since the graph-search problem increases in size.
- The set of motion primitives can be **computed offline**.
- **Tabulated values for computing heuristics** for distance between two states (e.g., representing the cost-to-go in free space).

Example: Lattice Planning (1/2)

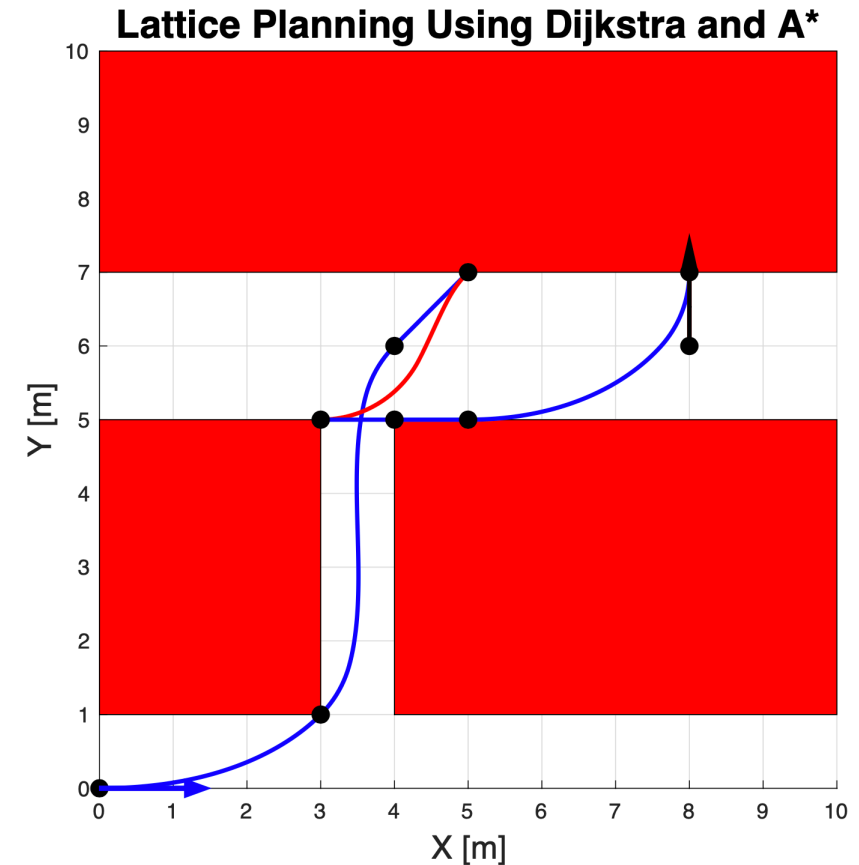


- The task is to **move from the blue arrow to the black arrow**, using the set of motion primitives shown on a previous slide.
- Application of the Dijkstra and A* algorithms from **Hand-in Exercise 1** gives the results in the plot to the right (blue lines – forward, red lines – reverse).



Example: Lattice Planning (2/2)

- Some observations in the results:
 - The finite set of motion primitives **implies restrictions on the motion.**
 - The **choice of graph-search strategy** leads to different solutions.
- Further studied in Hand-in Exercise 2.



Tools and Libraries for Motion Planning

The Open Motion Planning Library ⁶¹

Open Motion Planning Library

- **Library** containing several different state-of-the-art algorithms for **sampling-based motion planning**.
- A subset of the planners also support **optimal motion planning** and planning with **differential constraints**.
- Possible to interface with **visualization** and **collision-checking** software libraries.
- Homepage: <https://ompl.kavrakilab.org>

ROS Package MoveIT

- A ROS (Robot Operating System) package for **motion planning**.
- Includes, e.g., **interfaces to planners** from:
 - Open Motion Planning Library,
 - Search Based Planning Library
- **Visualization** is possible using the tool RViz.
- C++ and Python interfaces are available.
- Homepage: <https://moveit.ros.org>

Decoupled Approaches to Motion Planning

The material in the following section is slightly more advanced and can be considered as extra material in the course. The methods presented here will not be part of the hand-in exercises.

Decoupled Approach to Motion Planning

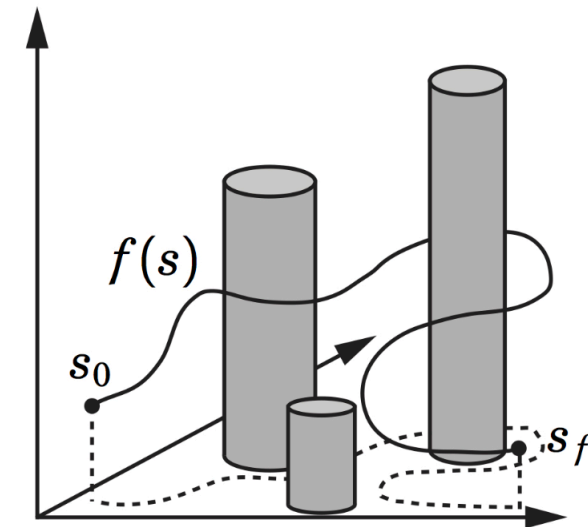
- The motion-planning problem can be **decoupled** into the following two phases to **reduce complexity**:
 - **path planning** and
 - **trajectory planning** (~velocity planning).
- The second problem is then often called **path-constrained trajectory planning**.

Parameterization of Vehicle Dynamics on Path (1/2)

- Assume that a **path** is described by a (two-times differentiable) function of a (scalar) **path coordinate**

$$q = f(s), \quad s \in [s_0, s_f], \quad q \in \mathbb{R}^n$$

- The idea is now to describe the vehicle dynamics using this additional **path-tracking requirement**.



Parameterization of Vehicle Dynamics on Path

- Using the **chain rule**, the following relations can be shown

$$\dot{q} = \frac{df}{dt} = \frac{df}{ds} \frac{ds}{dt} = f'(s)\dot{s},$$

$$\begin{aligned}\ddot{q} &= \frac{d^2f}{dt^2} = \frac{d}{dt} \left(\frac{df}{ds} \frac{ds}{dt} \right) \\ &= \frac{d^2f}{ds^2} \frac{ds}{dt} \frac{ds}{dt} + \frac{df}{ds} \frac{d^2s}{dt^2} \\ &= f''(s)\dot{s}^2 + f'(s)\ddot{s}\end{aligned}$$

$$\begin{aligned}(\quad)' &\sim \frac{d}{ds} \\ (\dot{\quad}) &\sim \frac{d}{dt}\end{aligned}$$

Example: Particle Motion in Two Dimensions (1/2)

- Assume that we have a **particle moving in a plane** like a 2D printer, where the **force** in each direction can be controlled

$$m_1 \ddot{q}_1 = u_1, \quad m_2 \ddot{q}_2 = u_2, \quad -1 \leq u_1, u_2 \leq 1$$

- Using the relations on the previous slide, it is obtained

$$m_1 (f'_1(s) \ddot{s} + f''_1(s) \dot{s}^2) = u_1,$$

$$m_2 (f'_2(s) \ddot{s} + f''_2(s) \dot{s}^2) = u_2$$

Example: Particle Motion in Two Dimensions (2/2)

- The path to be followed is a **straight line** described by

$$f_1 = k_1 s, \quad f_2 = k_2 s$$

- With this path and the considered vehicle dynamics, the problem is to **determine the path acceleration** $\ddot{s}$ (and thereby the control inputs u_1, u_2) such that

$$\begin{aligned} \frac{ds}{dt} &= \dot{s}, & -1 \leq m_1 k_1 \ddot{s} \leq 1, & & s(0) = s_0, & s(t_f) = s_f, \\ \frac{d^2 s}{dt^2} &= \ddot{s} & -1 \leq m_2 k_2 \ddot{s} \leq 1 & & \dot{s}(0) = 0, & \dot{s}(t_f) = 0 \end{aligned}$$

Transformation to Fixed Interval (1/3)

- The **time duration** of the motion is **unknown *a priori***, and thus the final time is part of the problem to compute.
- **Transformation** to a **fixed interval** (over values of the path coordinate), allows reduction of the **problem complexity**.

Transformation to Fixed Interval (2/3)

- For a general **second-order system**, the system dynamics can be written as follows

$$F(s, \dot{s}, \ddot{s}, u) = 0$$

- Introduce **new variables** according to

$$\alpha(s(t)) = \ddot{s}(t), \quad \beta(s(t)) = \dot{s}(t)^2$$

Transformation to Fixed Interval (2/3)

- Using the **chain rule again**, it is obtained that

$$\ddot{s}(t) = \frac{d}{dt} \sqrt{\beta(s(t))} = \frac{1}{2\sqrt{\beta(s(t))}} \beta'(s(t)) \dot{s} = \frac{1}{2} \beta'(s(t))$$

$$\Rightarrow \beta'(s) = 2\alpha(s)$$

- Assume now that the **cost function** to **minimize** in this motion-planning problem is the **time**. This cost can be written in the **path velocity** as follows

$$t_f - t_0 = \int_{t_0}^{t_f} dt = \int_{s_0}^{s_f} \frac{1}{\dot{s}} ds = \int_{s_0}^{s_f} \frac{1}{\sqrt{\beta}} ds$$

Optimization Problem for Path Tracking (1/2)

- With the transformations on the previous slides, the **time-optimal trajectory-planning problem for path following** can be stated as

$$\begin{aligned} & \underset{\alpha(s), s \in [s_0, s_f]}{\text{minimize}} && \int_{s_0}^{s_f} \frac{1}{\sqrt{\beta}} ds \\ & \text{subject to} && \beta' = 2\alpha, \quad F(s, \beta, \alpha, u) = 0, \\ & && u_{\min} \leq u \leq u_{\max} \end{aligned}$$

- The **solution** tells us **how fast we can go along the desired path**, while taking the motion constraints and control-input constraints into account.

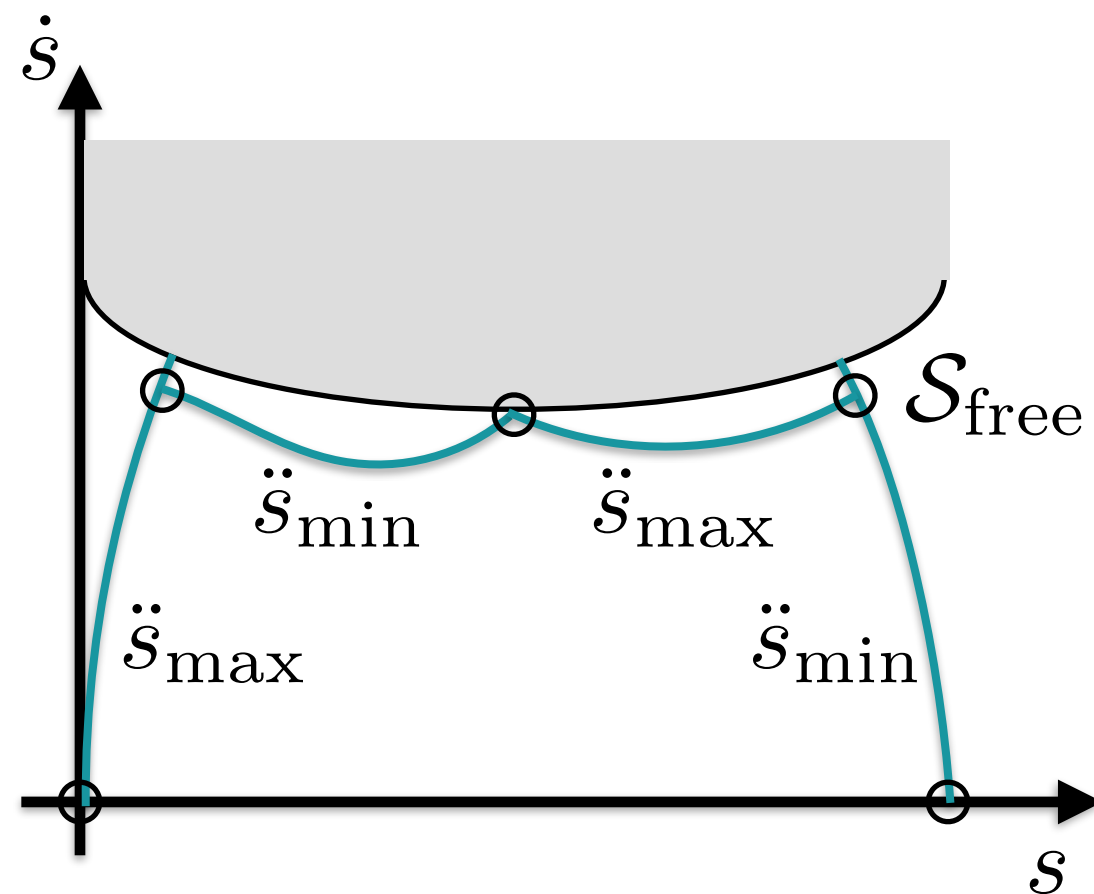
Optimization Problem for Path Tracking (2/2)

- The **optimization problem** can be solved using **numerical optimal control** (see Lecture 5) or by constructing the solution in the **phase plane** by integration (next slide).
- Note that the problem has been transformed such that
 - it is now over a **fixed interval** (time unknown *a priori*) and
 - the **state dimension is independent** of the number of degrees-of-freedom n in the considered vehicle dynamics.

Planning in the Phase Plane $s-\dot{s}$

- To compute the time-optimal solution, the path velocity $\dot{s}$ should be **as high as possible** along the path.
- The solution can be constructed by **integrating forward and backward** with maximum or minimum path acceleration $\ddot{s}$.
- The **allowed area in the phase plane** is described by

$$\mathcal{S}_{\text{free}} = \{(s, \dot{s}) : \ddot{s}_{\min}(s, \dot{s}) \leq \ddot{s}_{\max}(s, \dot{s})\}$$



Example: Time-Optimal Motion for the Particle Model (1/4)

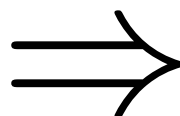
- Recall the previous example where the **particle is moving along a straight line** with constant limits on the forces. Let the parameters be

$$m_1 = 2, \quad m_2 = 1, \quad k_1 = 1, \quad k_2 = 1, \quad s_0 = 0, \quad s_f = 1$$

- Given the constraints on the inputs, the **time-optimal solution** should satisfy

$$\begin{aligned} -1 &\leq m_1 k_1 \ddot{s} \leq 1, \\ -1 &\leq m_2 k_2 \ddot{s} \leq 1 \end{aligned}$$

$$-1 \leq u_1, u_2 \leq 1$$



$$\ddot{s}_{\max} = \min_{i=1,2} \frac{u_i^{\max}}{m_i k_i} = \frac{1}{2},$$

$$\ddot{s}_{\min} = \max_{i=1,2} \frac{u_i^{\min}}{m_i k_i} = -\frac{1}{2}$$

Example: Time-Optimal Motion for the Particle Model (2/4)

- The time-optimal solution is of **bang-bang character**, i.e.,

$$\ddot{s} = \begin{cases} \ddot{s}_{\max}, & t_0 \leq t \leq t_1 \\ \ddot{s}_{\min}, & t_1 \leq t \leq t_f \end{cases}$$

- Integration two times** and utilization of boundary values and continuity of path coordinate and path velocity gives

$$s = \begin{cases} \frac{1}{4}t^2, & t_0 \leq t \leq t_1 \\ -\frac{1}{4}t^2 + \sqrt{2}t - 1, & t_1 \leq t \leq t_f \end{cases} \quad t_1 = \sqrt{2}, \quad t_f = 2t_1$$

Example: Time-Optimal Motion for the Particle Model (3/4)

- Recall the **differential relations** with respect to the path coordinate

$$\beta'(s) = 2\alpha(s), \quad \alpha(s(t)) = \ddot{s}(t), \quad \beta(s(t)) = \dot{s}(t)^2$$

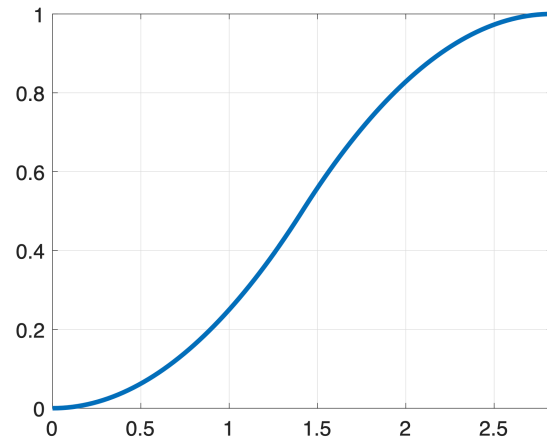
- Path acceleration** and **path velocity** then computed as

$$\alpha = \begin{cases} \ddot{s}_{\max}, & s_0 \leq s \leq s_1 \\ \ddot{s}_{\min}, & s_1 \leq s \leq s_f \end{cases} \quad \boxed{\beta'(s) = 2\alpha(s)} \quad \Rightarrow \quad \sqrt{\beta} = \begin{cases} \sqrt{s}, & s_0 \leq s \leq s_1 \\ \sqrt{(-s + 1)}, & s_1 \leq s \leq s_f \end{cases}$$

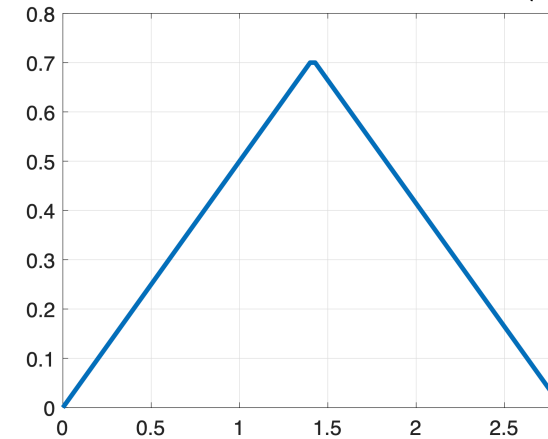
$$s_1 = \frac{1}{2}s_f, \quad s_f = 1$$

Example: Time-Optimal Motion for the Particle Model (4/4)⁷⁸

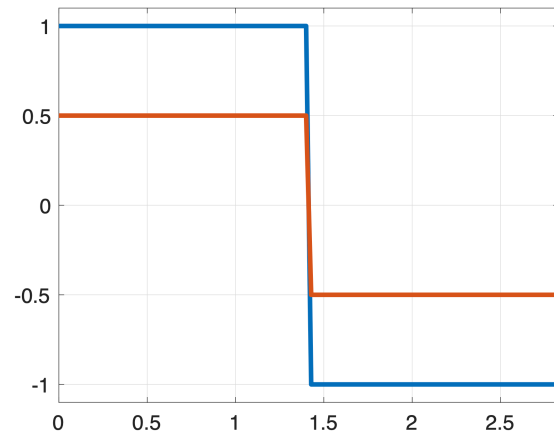
Path coordinate $s(t)$



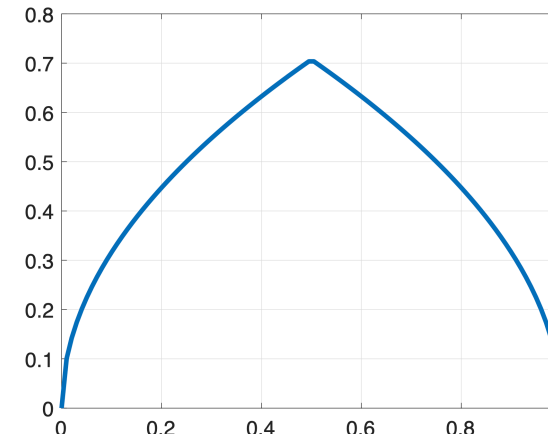
Path velocity $\dot{s}(t)$



$u_1(t)$ (blue), $u_2(t)$ (red)



Path velocity $\sqrt{\beta}(s)$



References and Further Reading

References and Further Reading (1/2)

All the following books and articles are not part of the reading assignments for the course, but cover the topics studied during this lecture in more detail.

- Bergman, K: "Exploiting Direct Optimal Control for Motion Planning in Unstructured Environments ", Ph.D. Thesis No. 2133, Div. Automatic Control, Linköping Univ., 2021.
- Bobrow, J. E., S. Dubowsky, & J. S. Gibson: "Time-optimal control of robotic manipulators along specified paths". *The International Journal of Robotics Research*, 4(3), 3-17, 1985.
- Dahl, O: "Path Constrained Robot Control", Ph.D. Thesis, Dept. Automatic Control, Lund Univ., 1992.
- Dubins, L. E., "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents", *American Journal of Mathematics*, 79(3), 497-516, 1957.

References and Further Reading (2/2)

- Karaman, S., & E. Frazzoli: "Sampling-based algorithms for optimal motion planning". *The International Journal of Robotics Research*, 30(7), 846-894, 2011.
- LaValle, S. M., & J. J. Kuffner Jr: "Randomized kinodynamic planning". *The International Journal of Robotics Research*, 20(5), 378-400, 2001.
- LaValle, S. M: *Planning Algorithms*. Cambridge University Press, 2006.
- Likhachev, M., & D. Ferguson: "Planning long dynamically feasible maneuvers for autonomous vehicles". *The International Journal of Robotics Research*, 28(8), 933-945, 2009.
- Ljungqvist, O: "Motion planning and feedback control techniques with applications to long tractor-trailer vehicles", Ph.D. Thesis, Div. Automatic Control, Linköping Univ., 2020.

www.liu.se